

分类号_____

U D C _____

学 号 MZ120220936

密 级 _____



硕士学位论文

(全日制专业学位)

基于 UVM 的 ARINC429 模块验证平台设计与实现

学 院： 信息工程学院
专 业 学 位 类 别： 电子信息硕士
专 业 学 位 领 域： 计算机技术
研 究 生： 查坤龙
校 内 指 导 教 师： 孙小兵 教授
校 外 指 导 教 师： 侯雅莉 南京晓庄学院
杨琪 西安微电子技术研究所
答 辩 委 员 会 主 席： 李斌 教授
答 辩 日 期： 2025 年 5 月 23 日

Design and Implementation of a UVM-Based ARINC429 Module Verification Platform

A thesis submitted to
Yangzhou University
in partial fulfillment of the requirements
for the degree of

Master of Electronic and Information Engineering

By

Kunlong Zha

Supervisor: Prof. Xiaobing Sun

Co-Supervisor: Yali Hou

Co-Supervisor: Qi Yang

Computer Technology

May 2025

目 录

第 1 章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 基于 UVM 的验证方法	2
1.2.2 ARINC429 模块的设计与验证	4
1.2.3 大语言模型在芯片领域中的应用.....	5
1.3 论文主要工作.....	6
1.4 论文组织结构.....	7
第 2 章 相关技术	9
2.1 UVM 验证方法学	9
2.1.1 UVM 验证平台的结构与组件	10
2.1.2 UVM 验证方法学的机制	11
2.2 ARINC429 总线协议	16
2.3 大语言模型与提示工程.....	18
2.4 本章小结.....	19
第 3 章 基于大语言模型的测试用例生成方法	21
3.1 引言.....	21
3.2 方法整体框架.....	22
3.3 方法关键步骤.....	23
3.3.1 覆盖率报告生成.....	23
3.3.2 覆盖率报告预处理.....	24
3.3.3 测试用例生成.....	25
3.4 实验设计.....	29
3.4.1 实验问题设计.....	29
3.4.2 实验过程.....	30
3.5 实验结果及分析.....	31
3.5.1 实验问题一	31
3.5.2 实验问题二.....	32
3.5.3 实验问题三.....	33
3.5.4 实验问题四.....	34
3.5.5 实验问题五.....	36
3.6 本章小结.....	37
第 4 章 基于 UVM 的验证平台设计与实现.....	39
4.1 ARINC429 模块设计	39

4.1.1 发送模块设计	40
4.1.2 接收模块设计	41
4.1.3 读写控制模块设计	42
4.1.4 寄存器组设计	42
4.2 验证平台设计	43
4.3 验证平台实现	45
4.3.1 Driver 实现	45
4.3.2 Monitor 实现	46
4.3.3 Sequencer 实现	47
4.3.4 Agent 实现	48
4.3.5 Scoreboard 实现	49
4.3.6 Reference Model 实现	50
4.3.7 测试用例生成实现	50
4.4 本章小结	51
第 5 章 仿真验证与结果分析	53
5.1 仿真环境	53
5.2 测试用例	54
5.2.1 数据发送功能测试	55
5.2.2 数据接收功能测试	56
5.2.3 寄存器基础功能测试	58
5.3 仿真结果分析	59
5.3.1 数据发送功能结果分析	59
5.3.2 数据接收功能结果分析	60
5.3.3 寄存器基础功能结果分析	61
5.4 覆盖率分析	61
5.5 本章小结	63
第 6 章 总结与展望	65
6.1 工作总结	65
6.2 未来展望	66
参考文献	69
附录	75

摘 要

随着航空电子系统集成度的不断提升，子系统间的通信可靠性面临严峻挑战。ARINC429 总线协议作为航空电子领域关键通信协议，对于设备的功能安全和高效运行至关重要。传统验证方法存在验证效率低和覆盖率不足的问题，难以应对日益复杂的协议需求。为了解决这一问题，本文提出了一种基于大语言模型的测试用例生成方法，旨在提升验证效率、覆盖率和全面性。具体而言，本文的研究工作集中在以下三个方面：

（1）提出了一种基于大语言模型的测试用例生成方法。与基于随机约束生成测试用例的方法相比，本文方法通过将设计代码和覆盖率报告等信息输入到大语言模型中，结合提示模板，实现高效的测试用例生成。实验结果显示，在完成相同验证目标下的，相较于基于随机约束生成的测试用例，基于大语言模型生成的用例显著提高了验证覆盖率，具体的，语句覆盖率提升了 8.33%，分支覆盖率提升了 9.45%，条件覆盖率提升了 8.05%，并且验证时间缩短了约 70%，从 4 小时减少至 1.3 小时。

（2）搭建了基于 UVM 的 ARINC429 模块验证平台。通过结合大语言模型生成的测试用例，本文成功搭建了一个自动化、灵活的 ARINC429 模块验证平台，能够根据反馈动态调整测试策略，确保 ARINC429 模块功能的全面验证。平台的高自动化性显著提升了验证效率，并通过优化测试用例确保了验证结果的准确性与全面性。

（3）完成了 ARINC429 模块的仿真验证与结果分析。通过结合手动设计的测试用例与大语言模型生成的测试用例，平台覆盖了更多的功能场景，提升了验证效率。在 Linux 环境下，使用自动化脚本进行仿真，经过对仿真波形结果和覆盖率报告的分析，并进行多轮回归测试，最终验证平台达到 100%的代码和功能覆盖率，实现了对 ARINC429 模块的完备性验证，证明了平台的有效性。

关键词：UVM；ARINC429；测试用例生成；验证平台；大语言模型

Abstract

With the continuous improvement in the integration of avionics systems, the reliability of communication between subsystems faces significant challenges. The ARINC429 bus protocol, as a critical communication standard in avionics, is essential for ensuring the functional safety and efficient operation of equipment. Traditional verification methods suffer from low efficiency and insufficient coverage, making them inadequate for addressing increasingly complex protocol requirements. To address these issues, this thesis proposes a large language model (LLM)-based test case generation method aimed at enhancing verification efficiency, coverage, and comprehensiveness. The research focuses on the following three aspects:

1. A large language model-based test case generation method is proposed. Compared to traditional UVM random constraint-based methods, this approach inputs design code, coverage reports, and other information into the LLM, combined with prompt templates, to achieve efficient test case generation. Experimental results show that, under the same verification objectives, the LLM-generated test cases significantly improve verification coverage: statement coverage by 8.33%, branch coverage by 9.45%, and condition coverage by 8.05%, while reducing verification time by approximately 70% (from 4 hours to 1.3 hours).

2. A UVM-based ARINC429 module verification platform is developed. By integrating LLM-generated test cases, an automated and flexible verification platform is successfully constructed. This platform dynamically adjusts testing strategies based on feedback, ensuring comprehensive validation of ARINC429 module functionalities. Its high automation significantly improves verification efficiency and ensures the accuracy and thoroughness of results through optimized test cases.

3. Simulation verification and result analysis of the ARINC429 module are completed. By combining manually designed test cases with LLM-generated ones, the platform covers more functional scenarios and enhances verification efficiency. In a Linux environment, automated scripts were used for simulation. Analysis of simulation

waveforms, coverage reports, and multiple rounds of regression testing confirmed that the platform achieves 100% code and functional coverage, validating the completeness of the ARINC429 module and demonstrating the platform's effectiveness.

Keywords: UVM; ARINC429; Test case generation; Verification platform; Large language model

第1章 绪论

1.1 研究背景与意义

在当今高新技术迅猛发展的时代，芯片技术以及集成电路产业的进步已然成为各个行业发展的核心推动力量，在人工智能、消费电子、航空航天等领域，芯片技术的创新与进步在技术发展过程中发挥着极为关键的作用^[1-3]。这些技术的突破，推动了信息技术的革新，还为各类复杂系统的设计与优化提供了有力的支持。在航空航天领域，航空电子系统正逐步发展成一个高度集成且复杂的通信网络，其中包含了飞行控制、推力控制、航电仪表、显示系统等多个关键子系统。这些子系统相互紧密协作，以此保证飞行器的安全性、精确性和实时性。

随着航空电子系统朝着自动化与智能化不断发展，飞行器设计和制造过程中所面临的挑战日益增多，特别是在保障航空电子系统同其他机载设备之间实现高效且稳定的通信。怎样保证数据可实时、准确且安全地进行传输，已然成为设计过程中的核心要点^[4]。就航空电子系统而言，数据总线协议作为信息传递的关键所在，发挥着极为关键的作用。怎样保证数据总线协议的稳定性和可靠性，成为现代航空电子系统设计里的一项关键任务^[5]。

ARINC429 总线协议属于航空电子系统里被广泛运用的通信标准中的一种，它所采用的差分串行单向传输方式，赋予了其低成本、易于维护以及良好兼容性等优点。随着标准持续完善，ARINC429 总线协议于全球航空领域得以广泛应用，在民用航空和商用运输飞机里应用极为普遍，甚至在部分军事装备中也有被采用^[6-9]。对于飞行器的各个子系统而言，ARINC429 协议的准确实现以及高效执行有着非常关键的意义，为达成这一目的，系统的硬件部分一般需要借助专门设计的芯片来执行协议的具体操作。

然而随着航空电子系统复杂程度持续攀升，对于 ARINC429 协议的支持而言，不仅需要有高效稳定的硬件实现，还得借助严格的芯片验证来保障协议在实际运行时的可靠性以及准确无误地执行^[10]。传统的芯片验证工作一般是利用

硬件描述语言来编写测试代码，开展模拟验证，以此保证芯片在不同场景中对于协议的执行契合设计要求。然而随着芯片设计变得日益复杂，传统验证方法的局限性渐渐凸显出来，在那些需要大量随机化数据以及穷举式仿真测试的场景里，传统验证方法的不足之处格外明显。

为了应对并解决这些问题，国内外的验证工程师和电子设计自动化（EDA）软件厂商始终在持续不断地进行探索，尝试找寻新的验证语言以及方法。在 2011 年，Accellera 推出了基于开源验证方法学（OVM）的通用验证方法学（UVM），其吸取了多种验证方法的优点并不断优化，目前已经成为芯片验证领域的标准，成为 IEEE 1800.2 标准的一部分^[11]。UVM 给出了一系列标准化、可扩展、面向对象的验证组件以及方法，支持从测试生成一直到结果分析的整个验证流程，大幅度提升了验证的效率与质量。

随着 UVM 验证流程趋向成熟，验证工程师们着手探寻更为高效的验证方式。近些年来，大语言模型于软件开发以及测试领域的应用收获了一定进展，在硬件描述语言领域也开始彰显出其强大潜力，比如大语言模型已然成功运用于自动生成 Verilog 代码、编写断言，并且在 EDA 工具脚本的生成方面发挥了作用^[12]。研究显示，大语言模型可以理解 Verilog 代码的语法、结构以及功能需求，可为芯片设计与验证过程给予有效支撑，提升了开发效率与准确性。

在上述背景之下，怎样借助 UVM 来构建针对运用 ARINC429 总线协议的模块验证平台，并且结合大语言模型去提升验证的效率以及准确性，这成为了本文研究的关键核心问题所在。本文会深入探讨怎样把大语言模型运用到芯片验证领域当中，借助一系列范围较为广泛的实验，对大语言模型在该领域的应用成效给予验证。期望本文的研究可为芯片验证工程师以及大语言模型研究者提供全新的思路，推动芯片验证方法实现创新与发展。

1.2 国内外研究现状

1.2.1 基于 UVM 的验证方法

在芯片验证发展的早期阶段，像 VMM 以及 OVM 这样的验证方法学为验证工作构建起了初步的框架，它们为验证方法的规范化发展做出了贡献，然而鉴

于芯片设计变得日益复杂，这些方法各自存在着不足之处，难以契合现代芯片验证的实际需求。为了应对这些问题，Accellera 在 2011 年推出了 UVM，这种方法学在 VMM 和 OVM 的基础之上给予优化，提供了更具模块化特点、更可重用的验证平台框架^[11]。

UVM 是基于 SystemVerilog 语言构建的，它充分呈现了面向对象设计的优势，让验证工作变得更为灵活且高效，并且借助组件复用提升了平台的可扩展性与可维护性。跟传统验证方法相较而言，UVM 借助高度模块化以及标准化的验证流程，极大程度地简化了验证环境的构建进程。它能让工程师复用已有的验证组件，有效缩短验证周期并提高验证效率，UVM 的出现，解决了传统验证方法在应对复杂设计时的局限性。

正如 Reddy 等人^[13]指出，传统验证方法面对日益复杂的芯片设计时局限性渐显，模块化设计方面存在不足，UVM 借助允许验证组件复用，大幅减少了构建环境所需的时间与精力，让工程师能把更多精力投入到验证计划与策略的制定中，有效提高了验证工作的效率与准确性。Gamboa 等人^[14]也强调，UVM 的标准化环境和可复用组件简化了验证工作，其约束随机测试方法能使功能覆盖率从传统定向测试的 65% 提升至 90% 以上，加快了芯片产品的上市进程。

UVM 被广泛用于芯片验证之后，相关研究探讨了其在不同芯片验证任务里的应用成效。CW Liao 等人^[15]针对基于 UVM 的方法在 SPI 协议验证中的应用展开研究，着重关注于可穿戴设备以及物联网设备方面的验证工作，此项研究构建了一个有可扩展性的 SPI 验证平台，对多个从设备协调以及错误处理场景给予验证，最终证实了 SPI 协议在实际设备中的功能以及可靠性。结果说明，验证平台的功能覆盖率为 83.33%，断言覆盖率达 100%，成功识别出边缘情况以及潜在的时序违规等问题，证实了 UVM 方法在 SPI 协议验证中的有效性，RA Prasad 等人^[16]探讨了 UVM 在高速通信芯片验证中的应用情况，发现 UVM 可切实缩短测试周期，同时提升了测试覆盖率以及系统稳定性。

虽然 UVM 在多个芯片验证工作里取得了成果，不过在生成测试用例的效率以及覆盖率这两方面，仍然要依靠大量人工参与，因此需要结合新的技术方法来提高验证效率与覆盖范围。

1.2.2 ARINC429 模块的设计与验证

ARINC429 总线协议作为航空电子系统中关键的数据传输标准，对其开展验证工作，对于保障系统有安全性、稳定性以及高效性十分关键，ARINC429 总线协议在民用航空、商用飞机以及部分军事装备当中有着广泛应用，像是飞行控制、导航系统、引擎监控等多个关键子系统都有它的身影。ARINC429 总线协议的可靠性会直接对航空器的整体性能以及飞行安全产生影响，保障其功能正确实现是航空电子系统设计的一个关键方面。

当前对于 ARINC429 总线协议的验证，主要是依靠手动设计测试用例以及硬件仿真等传统办法，一般是借助 FPGA 模拟、硬件仿真以及人工编写测试用例等形式来验证协议的正确性。

Kamaraju 等人^[17]在他们的研究里借助 FPGA 平台达成了 ARINC429 总线协议，并且结合 Verilog 语言开展模拟，设计以及手动编写了多个测试用例，之后凭借和预期结果作对比去验证协议的正确达成。Yan 等人^[18]借助设计并达成了多通道 ARINC429 信号的同步以及实时仿真，成功构建了硬件仿真环境，还凭借 FTP 把数据帧传送给仿真软件，验证了协议达成的正确性。

随着验证需求持续升级，以 UVM 为基础的验证方法学渐渐成为可提升验证效率以及准确性的有效工具。Ibrahim 等人^[19]借助 UVM 构建了 ARINC429 总线协议的验证平台，还提出了一种可自动生成 ARINC429 数据帧的算法，此算法让测试用例生成效率相比人工编写提高了 80%，大幅度提升了验证的自动化程度以及效率。UVM 方法学依靠标准化的验证环境，帮助工程师简化了验证流程，并且保证了高效且精确的验证结果。

阳一涵等人^[20]设计并实现了基于 UVM 的 ARINC429 总线接口模块验证平台，还创新性采用自动更新的聚合参数，规避了传统方法里手动更新参考模型参数的繁杂流程，提高了验证平台自动化程度，保证了验证结果独立性。李屹爽等人^[21]设计并验证了基于 UVM 的 APB-ARINC429 模块，此平台借助受约束的随机测试，把验证周期从传统方法的 4 周缩减到 1.5 周，提升了验证效率，缩短了开发周期，推动了芯片验证领域发展。

虽然当前基于 UVM 的 ARINC429 验证方法有了一定进展，不过随着芯片设计以及验证需求持续提高，怎样提升验证自动化程度、优化测试用例生成策略、

提高验证效率与准确性依旧是当前研究的关键方向。

1.2.3 大语言模型在芯片领域中的应用

随着芯片设计复杂程度持续攀升，传统验证方法遭遇了前所未有的挑战。在功能验证进程中，设计人员要对 RTL 代码展开深入细致的分析，还要手动编写数量众多的测试用例，以此保证设计的各个部分可获得充分的测试。不过随着设计规模不断拓展以及团队协作变得日益复杂，传统验证方法渐渐显现出效率低下、人工成本高昂、验证全面性欠缺等问题。如何提升验证的效率与准确性，已然成为芯片设计验证领域急需解决的关键问题。

在这样的背景之下，大语言模型的引入给芯片验证工作给予了新的角度以及解决办法。近些年来，像 GPT-4 这样的大语言模型，依靠其强大的自然语言处理能力，引发了各个行业的广泛关注，在芯片设计领域，它的潜力慢慢显现出来，把大语言模型运用到生成 Verilog 代码、编写测试用例以及自动化断言等任务当中，芯片设计与验证流程得到了优化^[22]。研究显示，大语言模型在硬件设计里的应用，在 RTL 代码生成的精细化进程方面，呈现出了很大的潜力。比如有些研究提出了多种办法，改善了 Verilog 代码生成的质量，展示了大语言模型怎样简化并提高硬件设计工作流的效率^[23-26]。

在芯片验证领域，大语言模型的应用并非只局限于代码生成，而是延伸到了范围更为广阔的验证任务当中，比如说，有一些研究对怎样利用大语言模型自动生成断言展开了探索^[27-28]，借助这些自动化生成的断言，设计人员在验证过程里可减少手动编写验证代码的繁杂工作，并且还提高了验证覆盖率以及准确性。以这样的方式，验证的自动化程度有了很大提升，减少了因为人为因素产生的误差，还切实提高了验证效率。

大语言模型的提示工程也为芯片设计与验证带来创新性解决方案。部分研究显示，大语言模型在芯片架构构思及细化阶段能发挥作用，面对复杂硬件架构时，模型可依据设计目标与约束条件，给出有效的设计建议，简化设计与验证的工作流程^[29-31]。此过程提升了设计效率，也让验证过程更精准高效。大语言模型与自动化工具相结合，有潜力在 EDA 流程中实现任务自动规划与执行，优化芯片设计与验证的整体效率^[32]。

1.3 论文主要工作

本文围绕基于 UVM 的 ARINC429 模块验证平台的设计与实现展开，研究的主要内容包括基于大语言模型生成测试用例的策略、ARINC429 模块验证平台的构建、在验证平台搭建完成后，通过结合手动设计与大语言模型生成的测试用例，进行了 ARINC429 模块的仿真验证，最终达到了 100%的功能和代码覆盖率，验证平台的有效性得到了充分证明。具体工作如下：

（1）基于大语言模型的测试用例生成方法：传统的验证方法通常依赖人工生成测试用例，这种方式不仅验证效率低，而且覆盖率有限，难以应对复杂协议的验证需求。为此，本文提出了一种创新的基于大语言模型的测试用例生成方法。该方法通过将设计代码、覆盖率报告等信息输入到大语言模型中，结合自定义的提示模板，生成精准且全面的测试用例。模型能够根据验证过程中的反馈与覆盖率分析，自动优化生成的测试用例，从而不断提升验证覆盖率和验证效果。实验结果表明，与基于随机约束生成的测试用例相比，基于该方法生成的测试用例能够覆盖更多的功能场景，显著提高了验证覆盖率，提升了验证的全面性与效率。

（2）基于 UVM 的 ARINC429 模块验证平台：在提出基于大语言模型的测试用例生成方法的基础上，本文成功搭建了一个基于 UVM 的 ARINC429 模块验证平台。通过自动生成的测试用例，该平台能够在不同的验证阶段，根据反馈动态调整测试策略，确保 ARINC429 总线协议各项功能的全面验证。该平台的高度自动化和灵活性显著提升了验证效率。通过与大语言模型生成的测试用例结合，平台能够在验证过程中不断优化测试用例，确保验证结果的准确性与全面性，显著提高了验证的效率和验证的质量。

（3）ARINC429 模块的仿真验证与结果分析：在验证平台搭建完成后，本文进一步进行了 ARINC429 模块的仿真验证工作。对 ARINC429 协议的各个功能模块和时序要求进行了详细规划，针对每个功能手动策划了相应的测试用例，并结合大语言模型生成测试用例。通过这种组合方式，验证平台能够覆盖更多的功能场景，提升验证效率。仿真结果显示，经过多轮回归测试后，验证平台达到了 100%的代码和功能覆盖率，实现了对 ARINC429 模块的完备性验证，

验证了平台的有效性。

1.4 论文组织结构

本篇论文有六个章节，如下所示。

第一章：绪论。本章主要介绍了论文的研究背景与意义，分析了国内外相关领域的研究现状，阐述了本课题的来源和研究的主要内容。同时，本章还概述了论文的整体结构。

第二章：相关技术理论。本章介绍了 UVM 验证方法学、ARINC429 总线协议、提示工程和大语言模型。通过对相关技术的系统梳理，为后续的研究和实验工作奠定了理论基础。

第三章：基于大语言模型的测试用例生成方法。本章探讨了如何利用大语言模型生成测试用例，分析了传统方法生成测试用例的局限，并提出了通过大语言模型自动生成高质量测试用例的方法。实验表明，这种方法能够显著提高测试覆盖率和验证效率。

第四章：基于 UVM 的验证平台设计与实现。本章介绍了基于 UVM 框架搭建的 ARINC429 验证平台的设计与实现。详细描述了验证平台的整体架构、模块划分，并探讨了大语言模型如何与平台集成，通过动态调整测试策略提升测试覆盖率和验证效率。

第五章：仿真验证与结果分析。本章基于前述验证策略与平台设计，展示了 ARINC429 模块的仿真验证过程。通过分析仿真波形与覆盖率数据，评估了大语言模型生成的测试用例在验证中的效果。结果表明，结合手动和自动生成的测试用例，验证平台达到了 100%的代码和功能覆盖率，提升了验证的效率与全面性。

第六章：总结与展望。本章对论文的主要工作进行了总结，并对未来工作进行了展望，提出了进一步优化的可能路径和研究思路。

第2章 相关技术

2.1 UVM 验证方法学

UVM (Universal Verification Methodology) 是一种开源的硬件验证方法学, 建立在 SystemVerilog 语言^[33]基础之上, 凭借构建结构化且可重复使用的验证平台, 来提高数字集成电路验证的效率和质量。UVM 作为行业里标准的验证方法, 为验证工程师给予了一个高效、灵活、还可以扩展的框架, 帮助他们应对日益复杂的集成电路设计方面的挑战。UVM 的架构来源于 OVM (Open Verification Methodology), 并且吸收了 VMM (Verification Methodology Manual) 的方法学精华, 还添加了不少创新内容, 最终成为数字设计验证的关键工具^[34]。

在 UVM 中, 前端功能验证所追求的目标在于借助仿真测试, 来保证待测设计的寄存器传输级代码契合预期功能, 防止设计缺陷于后期开发阶段引发高额成本。UVM 引入了面向对象的编程理念, 并且结合了 SystemVerilog 语言的特性, 像约束随机生成、覆盖率分析以及分层验证等, 提供了一整套完备的类库与机制。这让 UVM 可使验证流程得到简化, 而且还可以提升验证的准确性以及可靠性。

UVM 借助标准化的验证平台架构以及强大的类库支持, 帮助验证工程师迅速搭建起可扩展且可重复使用的验证环境, 其运行机制与组件的设计, 让验证平台变得灵活多样, 同时拥有很高的可维护性^[35]。UVM 的成功运用, 依靠其丰富的类库和架构设计, 也得益于它对多种关键机制的实现。这些机制囊括控制验证流程的 Phase 机制、实现组件重用的 Factory 机制、生成测试序列的 Sequence 机制、协调仿真各阶段的 Objection 机制, 以及支持数据传递的 Config_db 机制等^[36-39]。

接下来, 本章会对这些机制以及它们于 UVM 验证平台里的具体运用展开深入的探讨, 以此帮助理解 UVM 是怎样借由这些核心功能, 来优化验证流程并且提高工作效率的。

2.1.1 UVM 验证平台的结构与组件

UVM 验证平台的基本架构是基于 UVM 类库中的组件构建的，它为验证环境的设计提供了一种层次化的框架。此架构的核心思想是把复杂的验证任务给予模块化，借助标准化的构建流程，让验证工作能更具效率、更为灵活并且便于复用。UVM 的基本架构是由多个关键组件所构成，这些组件肩负着不同的职责，包括驱动器（driver）、监视器（monitor）、代理（agent）、序列传输器（sequencer）、得分板（scoreboard）和参考模型（reference model）等^[40]。

如图 2-1 所示，UVM 的基本架构呈现出了这些组件彼此之间的关系以及层次结构。在此架构中，验证环境（env）处在最顶层的位置，其承担着协调并且管理其他组件的职责。各个组件依靠标准接口实现连接，以此来保证数据流以及控制流可顺利地进行传递。

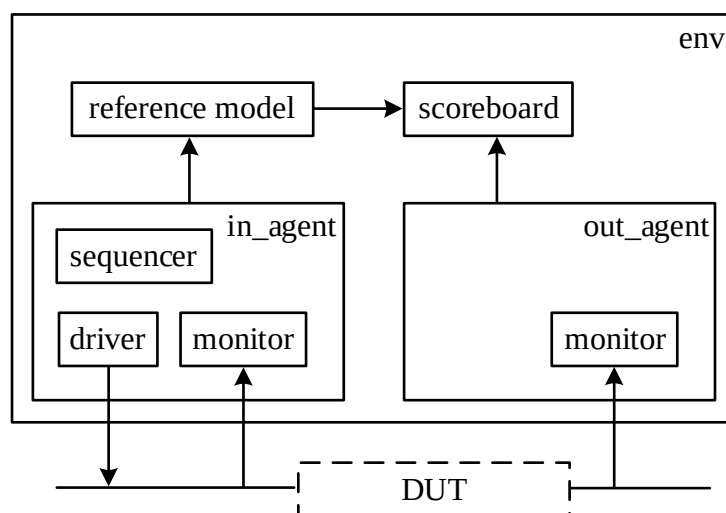


图 2-1 UVM 基本架构

以下为各个组件的功能简介：

driver: 驱动器组件承担着从 `uvm_sequencer` 接收事务的职责，并且会依据时序要求朝着 DUT（Design Under Test）发送激励信号。驱动器所发挥的作用是把测试用例转变为实际的硬件信号，以此保证验证环境可模拟出 DUT 的工作状态。

monitor: 监视器组件的主要职责在于，针对待验证设计即 DUT 的输入以及输出信号展开监控工作，并且把这些信号转变成为事务级的数据，以便用于后续的分析。监视器于验证进程里不会使 DUT 的状态产生改变，仅仅发挥着观察

以及记录的功用。

sequencer: 序列传输器组件承担着生成并控制测试激励发送顺序的职责，它会和驱动器共同协作，以此来保证测试激励依照设定好的顺序给予发送。序列传输器有模拟各类事务传输场景的能力，可以对多组事务的发送顺序加以控制，在复杂的 SoC 验证当中发挥着关键作用。

agent: 代理组件整合了驱动器、监视器以及序列传输器等多项功能，其作用在于协调 DUT 接口和其他组件之间的交互。一般每个代理组件都对应着一个特定接口，它可以对与该接口相关的所有验证任务进行有效管理。

scoreboard: 得分板组件负责检查 DUT 的输出是否符合预期，它通过比较监视器捕获的实际输出与参考模型的预期输出，确保设计功能的正确性。一旦发现不匹配的情况，得分板会报告错误。

reference model: 参考模型一般是对 DUT 功能的一种抽象模拟，它可给出预期的输出结果，以此辅助验证策略，减少错误定位所耗费的时间，同时帮助分析验证的覆盖率情况。

2.1.2 UVM 验证方法学的机制

UVM 的核心要点是借助一系列管理机制，切实有效地协调并控制验证平台的运行状况，其中 **Factory** 机制承担着组件的动态创建以及替换工作，**Phase** 机制用于精准把控仿真过程中的各个阶段，**Sequence** 机制负责管理测试序列的生成以及执行情况，**Objection** 机制保证仿真进程可正确启动与结束，**Config_db** 机制提供灵活的配置共享以及传递功能^[41-45]。依靠这些机制的相互协作，UVM 验证方法学可高效管理验证环境，提高验证的自动化程度与精确性。

(1) **Factory 机制:** 在 UVM 框架中，**Factory** 机制属于一种核心设计模式，它为验证工程师给予了灵活且可扩展的对象创建以及管理办法。借助 **Factory** 机制，验证人员可动态地创建验证环境中的组件实例，并非要在编译时就确定对象类型。**Factory** 机制的关键功能有类型注册、动态对象创建、类型覆盖以及配置管理。类型注册允许运用 `type_id::create()` 方法把对象类型注册到工厂中，在运行时创建相应的实例，动态创建则可在不同测试需求之下按需生成不同的组件，并非依赖于编译时的静态配置。类型覆盖功能让工程师可以在测试运行时替换组件的默认类型，以适应不同的测试场景。另外 **Factory** 机制还凭借配置管

理保证组件可依据需求进行灵活配置，并且提供了像克隆和替换等高级功能，提升了验证环境的复用性以及可维护性。Factory 机制提高了验证平台的灵活性与扩展性，还使得测试环境的创建以及管理更加高效和简便。

(2) Phase 机制：在 UVM 框架中，Phase 机制作为控制验证环境中各个组件执行顺序的核心机制而存在。它把整个仿真过程划分成一系列阶段，每一个阶段都有着特定的功能以及目标，依靠严格的执行顺序去保证仿真过程有有序性与高效性。Phase 机制让仿真任务可依照计划顺利开展，还可以有效地协助验证工程师在复杂的验证环境中精准地控制任务执行的时机与顺序。

UVM 里的 Phase 机制可把仿真过程划分成多个阶段，就像图 2-2 所展示的那样，在仿真开始以前每个阶段都会被预先设定好，来完成特定的初始化或者配置工作。这些阶段被分成了两类，分别是 function phase 和 task phase。function phase 不会耗费仿真时间，主要用来做初始化以及配置等操作，而 task phase 是会消耗仿真时间的阶段，一般用于执行激励、监控以及数据采集等任务^[46]。Phase 机制借助阶段的细致划分组织了验证环境，同时凭借时间和空间的界定，保证了验证任务可准确按时执行。

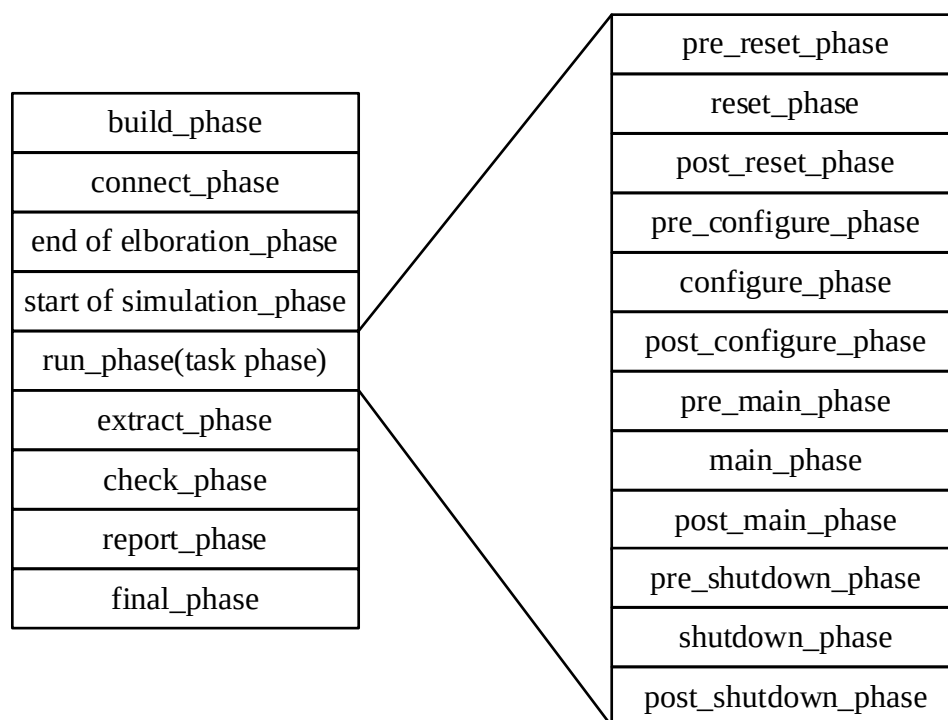


图 2-2 UVM 的 Phase 机制

在 UVM 中，所有的验证活动都是在这些不同的 Phase 中进行的，常见的主要阶段包括 Build Phase、Connect Phase、End of Elaboration Phase、Start of

Simulation Phase、Run Phase、Extract Phase、Check Phase、Report Phase 和 Final Phase。每个阶段的执行都有其明确的目标和任务，而这些阶段的组合则共同组成了一个完整的仿真周期。在仿真执行过程中，验证工程师不需要手动干预，UVM 会自动根据阶段的顺序和依赖关系执行各项任务，如表 2-1 所示。

表 2-1 UVM 的 Phase 机制

phase	函数/任务	执行顺序	功能
build	函数	自顶向下	创建和配置测试平台的结构
connect	函数	自底向上	建立组件之间的连接
end_of_elaboration	函数	自底向上	测试环境的微调
start_of_simulation	函数	自底向上	准备测试环境的仿真
run	任务	自底向上	激励设计
extract	函数	自底向上	从测试环境中收集数据
check	函数	自底向上	检查任何不期望的行为
report	函数	自底向上	报告测试结果
final	函数	自底向上	完成测试活动结束仿真

Build Phase 作为仿真开启的首个阶段，主要发挥着创建验证环境里各类组件的作用，像 **agents**、**drivers** 以及 **monitors** 等。此阶段并不会产生仿真时间的损耗，其重点任务在于对验证环境中的所有组件进行实例化以及初始化操作，为后续的验证活动给予必需的组件支持。紧随其后的 **Connect Phase** 阶段，承担着连接这些组件相互间接口的职责，以此保障各个组件之间可实现数据的传递。该阶段同样不存在仿真时间的消耗情况，其以便保证组件之间的连接精准无误，让后续的仿真操作可以顺利开展。

进入到仿真的核心阶段时，**End of Elaboration Phase** 和 **Start of Simulation Phase** 会进行一些最后的准备工作。在 **End of Elaboration Phase** 阶段，UVM 会打印出验证环境的具体结构信息，同时检查组件连接是否准确无误，紧接着，**Start of Simulation Phase** 将进行初始化配置，以此保证所有测试可在正确的初始条件下启动。

而 **Run Phase** 则是所有阶段中最重要的一個阶段，它是唯一一个会消耗仿真时间的阶段^[47]。在此阶段，验证环境中的各个组件着手执行实际的仿真任务，像激励生成、信号驱动以及数据采集等操作。这个阶段依据预先设定的测试用例来对设计展开验证，同时还会监测设计的行为，以此保证仿真可按照预期进行并收集到有效的测试数据。

当仿真执行结束之后，UVM 会进入到后处理阶段。首先是 **Extract Phase**，此阶段的主要职责是从仿真进程里提取数据，对仿真结果展开分析，生成如覆

盖率等相关报告。紧接着，Check Phase 会对仿真结果给予检验，以此保证仿真输出可符合预期。在这个阶段之中，验证工程师会将实际输出和预期结果进行对比，查看 DUT 是否依照预定功能来执行。

最后，Report Phase 和 Final Phase 是仿真结束时的收尾工作。在 Report Phase 阶段，UVM 会将各类信息汇总起来生成测试报告，其中囊括了覆盖率报告、错误日志以及仿真总结等内容，以此用来方便后续的验证评估工作。在 Final Phase，系统会开展资源释放、文件关闭等一系列清理工作，以此保障仿真顺利结束。

Run Phase 是唯一包含多个小阶段的任务阶段，有 Pre-Set Phase、Reset Phase、Post-Reset Phase 等。各小阶段负责处理仿真过程中不同任务，像复位、初始化配置、执行主要测试等，所有这些小阶段会依次执行，保证仿真流程每一步都获有效处理。

UVM 的 Phase 机制给仿真环境给予了严格的结构化管理，依靠此机制，验证工程师可清楚地知晓每个阶段的执行次序，以及每个阶段所开展的特定任务。并且 Phase 机制也赋予了测试高度的可定制性，工程师可依据不同的测试需求去调整不同阶段的执行顺序或者引入新的阶段，以此来适应持续变化的验证需求。

UVM 的 Phase 机制借助定义多个生命周期阶段，让验证环境拥有清晰、可控制的初始化、运行以及退出流程。在 run_phase 阶段，UVM 运用隐式的并行执行模型即 Parallel Process Model，所有组件的 run_phase 以线程形式并发执行，此机制提升了验证环境的并发处理能力。比如多个 Agent 的 Driver、Monitor 可并行执行，分别处理独立的激励驱动与信号监测，缩短验证时间，提高数据吞吐能力。此外，在激励生成、数据采集和结果分析等任务之间，还可借助 phase jump 和 objection 机制进行灵活调度，优化验证流程，降低阶段切换导致的同步等待。这种并行机制可提升验证的运行效率，还可提高验证环境针对高复杂度场景的响应能力，以支持多协议、多通道以及多场景的并行验证。

(3) Sequence 机制：在 UVM 中，Sequence 机制属于生成以及管理验证激励的关键办法^[48]。Sequence 是一类独立于平台的类，它的功能是产生 transaction 并把它传送给 Sequencer，接着由 Sequencer 把这些事务交给 Driver 去执行。展开来说，Sequence 生成的 transaction 会被送到 Sequencer，Sequencer 承担着协调与控制事务流向的职责，最后把它传递给 Driver 去驱动待验证的设计单元。这样的分离架构让验证工程师可灵活把控激励生成与驱动的进程，而不用对验证

平台自身进行修改，提升了平台的通用性和可复用性。

在一个 Sequencer 负责处理多个 Sequence 的情况下，有时需要引入 Virtual Sequencer 来对多个 Sequencer 的操作加以管理。Virtual Sequencer 作为一个中间层，它会协调多个 Sequencer 的事务生成，并且保证它们与底层 Driver 之间的连接不会出现冲突。借助这样的方式，多个 Sequence 可并行执行，而且每个 Sequence 都可以独立产生其特定的事务，这样的设计让 UVM 测试平台变得更加高效，还可支持更为复杂的测试场景。

Sequence 机制的优势在于它的高度模块化和灵活性，这使得测试环境可适应不同的验证需求，验证工程师可借助定制不同的 Sequence，去模拟多种复杂的激励场景，以此提高测试覆盖率，并且验证设计功能。

(4) Objection 机制：在 UVM 中，Objection 机制是控制仿真流程的关键工具。它借助“异议”概念，让测试平台中的组件能提出或者撤销运行请求，精准管理仿真执行。raise_objection 操作是用来提出异议，说明仿真该持续进行，drop_objection 操作则是用于撤销异议，意味着仿真可结束。借由这种办法，Objection 机制给出了一种灵活的控制途径，促使验证过程可以在多个层面实现精确管理。

UVM 建议在最顶层的 Virtual Sequence 里对 Objection 实施集中管理，如此一来可以保证仿真在整个验证流程中一直维持有序状态。Virtual Sequence 承担着协调与调度各个组件 Objection 状态的任务，以此保证仿真可依照预期的顺序启动与结束。借助这种集中控制的办法，可切实防止因局部管理 Objection 而致使的仿真中断或者延迟情况出现。

在实际运用当中，Objection 机制大多时候被用于对仿真流程里的 task phase 加以控制。UVM 在进入某一个 phase 的时候，会去检查是否有 Objection 被提出。要是没有任何 Objection 被提起，UVM 就会马上跳过当前 phase，直接进入到一个 phase；要是 Objection 被提出，那么仿真就会等待这些 Objection 被撤销之后才会继续执行下去。如此一来，UVM 可精准地把控仿真中各个阶段的执行顺序，防止出现无序执行或者提前结束仿真的情况。

Objection 机制对调试过程很有帮助，能让其变得清晰且高效。有些情况下仿真没按预期结束，工程师可依靠追踪提出 Objection 的组件以及撤销 Objection 的组件，找出仿真延长的根本原因。该机制提升了仿真控制灵活性，让测试流程更可控。

Objection 机制是 UVM 框架中不可或缺的一部分，它借助对仿真里关键阶段加以控制，以此提高验证效率，保证仿真可以依照预先设定的步骤顺利开展，防止出现不必要的延迟或者中断^[49]。借助合理运用这个机制，验证工程师可有效管理仿真流程，提升整体测试的精确性与效率。

(5) Config_db 机制：在 UVM 中，Config_db 是一种重要的机制，其作用是在不同的组件之间实现配置信息的共享。它借助一个全局数据库来对配置信息给予管理，能让组件在不凭借显式传递参数的情形下，实现对配置的访问以及修改。这样的机制可促使验证环境的设计朝着更加灵活且模块化的方向发展，降低了组件之间的耦合程度。

Config_db 的关键功能借助 set 和 get 方法达成数据的存取操作。set 方法可在测试开始之前把配置信息写入数据库之中，get 方法则可让组件在运行期间读取这些配置信息。组件并非直接依靠其他组件去获取参数，而是借助 Config_db 动态地获取配置信息，以此保证验证环境有高度的模块化^[50]。

比如当有多个模块都需要共享同样的配置信息，像时钟频率这类信息时，Config_db 可在顶层实施配置操作，而其他模块借助 get 方法来进行访问。采用这种方式可避免硬编码的情况出现，还可以减少在每个组件里重复传递参数的必要性，保证了验证环境有灵活性与可扩展性。

借助这种机制，Config_db 可提供一种较为简洁的途径来对测试环境里的配置参数加以管理。它减少了模块之间的直接依赖情况，同时还提升了验证平台的可维护性能以及重用性能。

2.2 ARINC429 总线协议

ARINC429 总线协议属于串行通信协议范畴，在航空电子设备间的数据传输方面有着广泛应用。该协议是由美国的航空电子标准组织 ARINC 于 1977 年所制定，其目的在于契合航空电子系统里对于数据传输所提出的高可靠性、低功耗以及高抗干扰能力等需求。ARINC429 采用的是单向数据传输方式，借助一对差分信号线也就是数据线和地线来开展通信工作。在航空电子系统当中，ARINC429 总线协议大多时候被应用于飞行控制、导航、通信以及引擎监控等关键领域。协议的帧格式如图 2-3 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P	SSM	MSB DATA LSB																				SDI	LABEL								

图 2-3 ARINC429 帧结构图

ARINC429 总线协议在进行数据传输的时候采用的是 32 位的数据帧格式，在这个 32 位的数据帧格式里，第 32 位充当奇偶校验位，其作用是保障数据传输可完整无误；第 30 位至 31 位构成信号状态矩阵，借助这个矩阵可标识信号的有效性；第 11 位至 29 位作为数据位，承担着传输数据的功能；第 9 位至 10 位是源/目的标记位，依靠它可以标识信号的源目的位置；第 1 位至 8 位为标识，可起到标识信号的作用。

在传输速率方面，ARINC429 总线协议有支持多种不同传输速率的能力，像 12.5kbps 以及 100kbps 等速率都在其支持范围内。正是因为这些各异的速率设置，ARINC429 总线协议才可依据不同设备与系统的具体需求，挑选出恰当的传输速度，如此一来，能保证数据传输的可靠性，又能提高系统的灵活性与适应性。

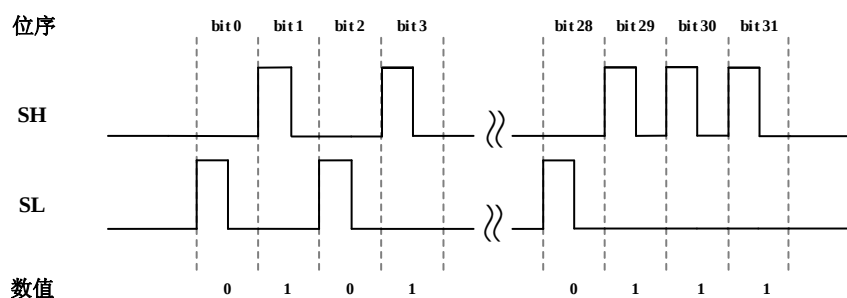


图 2-4 ARINC429 双极性归零码示意图

图 2-4 呈现出了 ARINC429 总线协议里所运用的双极性归零码的波形情况。在这种特定的编码方式当中，每个数据位是由两个连续出现的脉冲给予表示的，其中一个为正脉冲，另一个是负脉冲。脉冲所有的宽度被用于区分逻辑“1”以及逻辑“0”：正脉冲代表着逻辑“1”，而负脉冲则代表着逻辑“0”。ARINC429 总线协议采用的是差分信号传输方式，它拥有较强的抗干扰能力，就算是处于复杂的电磁环境里面，也依旧可保障数据的可靠传输，这种编码方式的设计所有的优势是，可同时进行数据和时钟信息的传输，以此保证每个数据位都有一个精确的时间间隔与之相对应。图中展示的波形涉及了两个 32 位数据字，每个数据字是由 16 个正脉冲和 16 个负脉冲构成的，它们分别对应着高电平以及低电平。依靠对波形进行观察，可以依据脉冲的宽度逐个解码数据字

的二进制值，比如图 2-4 中的第一个脉冲是负脉冲，bit0 为 0，紧接其后的正脉冲说明 bit1 为 1，按照这样的方式依次类推。

2.3 大语言模型与提示工程

大语言模型（LLM）是基于深度学习技术的自然语言处理（NLP）模型，能够处理和理解大规模文本数据。这些模型通常由数十亿到数千亿个参数构成，通过大规模的数据集进行训练，从而具备高度的语义理解和生成能力。随着计算能力得到提升以及数据资源不断积累，大语言模型在近些年于各个领域当中取得了一定进展，在自然语言生成、机器翻译、情感分析、文本摘要等任务方面有着较为出色的表现^[51-55]。

大语言模型的核心技术基于 Transformer 架构，该架构由 Google 在 2017 年提出，并迅速成为 NLP 领域的主流模型^[56]。Transformer 模型的主要创新点在于其自注意力机制（self-attention mechanism），这一机制可让模型有效地捕获长距离的依赖关系，在处理长文本时相较于传统的递归神经网络（RNN）更具优势。此外，Transformer 的并行计算能力显著提高了训练效率，推动了大规模模型的训练和应用。

目前，主流的大语言模型包括 OpenAI 的 GPT 系列^[57]、Google 的 BERT^[58]、T5^[59]和 PaLM^[60]、Meta 的 OPT^[61]等。以 GPT-4 为代表的生成式预训练模型采用无监督学习进行预训练，然后通过微调（fine-tuning）优化特定任务，使得大语言模型在各种 NLP 任务中展现出优异的性能。例如，GPT-4 不仅能够进行文本生成，还可以用于图像识别、代码生成、对话系统等多种任务。

随着大语言模型的不断发展，针对其生成内容的控制和优化需求日益增加，提示工程（Prompt Engineering）应运而生。提示工程指的是借助设计以及优化输入或者“提示”，以此引导大语言模型得出期望响应的过程，它作为一项新兴技术，在引导 LLM 生成精准且有相关性的输出这一方面发挥了十分关键的作用^[62]。

提示工程的关键目的在于凭借构建有效的提示，促使语言模型可产出更为精准且实用的回应。开发者借助合理的提示设计，可对语言模型的输出加以定制，依照特定任务的需求来优化其性能。提示设计在提升语言模型表现的也为新应

用场景的探寻给予了便利，推动了人工智能领域对话式人工智能方向的研究与发展。

提示工程在自然语言处理（NLP）任务中得到了显著应用。它借助优化提示结构，让 LLM 能生成更符合特定上下文且质量较高的响应，提高了模型在实际运用中的有效性与准确性。例如，通过为少量样本学习（few-shot learning）设计优化提示，能提升 LLM 在特定任务中的表现，这对于开发更高效的聊天机器人、虚拟助手及领域特定的 AI 工具具有重要意义^[63-66]。

在硬件验证这一领域当中，提示工程所起到的作用不容小觑。借助精心设计的提示模板，开发者可以引导 LLM 自动产出契合验证需求的测试用例，对测试覆盖率给予优化。如此一来，测试用例生成的效率得到了提升，同时也为验证策略的优化开拓了新的思路，对解决传统验证方法里人工干预过多以及测试覆盖率不足等问题颇有帮助。

随着提示工程这项技术持续发展，它会帮助解锁大语言模型的全部潜能。凭借可快速且准确地生成特定语言输出，提示工程可为各个行业带来更为高效以及智能化的工作流程。在未来，那些精通提示工程的技术人员或许会迎来新的就业机遇，而这一技术的进步也会推动更多高效且易用的用户界面展开开发，促使 LLM 的应用变得更加普及^[67-69]。

2.4 本章小结

本章主要介绍了 UVM 验证方法学、ARINC429 总线协议、大语言模型以及提示工程的背景和应用，为后续章节的研究与应用奠定了理论基础。

第3章 基于大语言模型的测试用例生成方法

3.1 引言

在芯片设计以及验证进程当中，测试用例的生成以及验证覆盖率的提高是保证芯片功能与性能可达到预期的关键要点。随着集成电路与总线协议的复杂程度持续攀升，传统的手动测试用例生成方式面临着效率不高、覆盖率不完整以及存在漏测等状况。在大规模的系统设计里，人工编写的测试用例大多时候难以覆盖所有可能出现的设计路径，容易引发验证盲点。这会增加验证所需的时间，而且还可能对最终设计的质量以及可靠性造成影响。

为了应对这些挑战，研究人员提出了多种方法来优化测试用例生成和验证覆盖率。一些传统方法依赖于基于规则的启发式生成策略，通过人工设计覆盖率标准来指导测试用例的编写。然而，这些方法在面对日益复杂的设计时，难以保证验证的全面性和效率。近年来，随着深度学习和自然语言处理技术的快速发展，基于大语言模型的自动化验证方法应运而生。这些方法能够利用大语言模型强大的上下文理解和生成能力，自动生成具有高覆盖率和针对性的测试用例，显著提升验证效率和准确性。

本文提出了一种基于大语言模型的测试用例生成方法，凭借自动化测试用例生成及覆盖率优化，解决传统验证方法局限，设计特定提示模板，引导大语言模型生成覆盖语句、分支与条件等多种验证目标的测试用例。此过程中，大语言模型验证方法可自动生成高质量测试用例，还可以借反馈机制持续优化验证结果，保证各验证目标充分测试。和传统人工编写测试用例方法相比，大语言模型验证方法可较大提升验证覆盖率，缩短验证周期，减少人工干预。

因此，本章会详细介绍大语言模型的测试用例生成方法，探讨其在测试用例生成方面的应用，以及在覆盖率优化和验证效率提升方面的应用，凭借实验验证该方法的有效性与优势。

3.2 方法整体框架

基于大语言模型的测试用例生成方法如图 3-1 所示。该方法包含了三个主要阶段，分别是覆盖率报告生成阶段、报告预处理阶段和测试用例生成阶段。

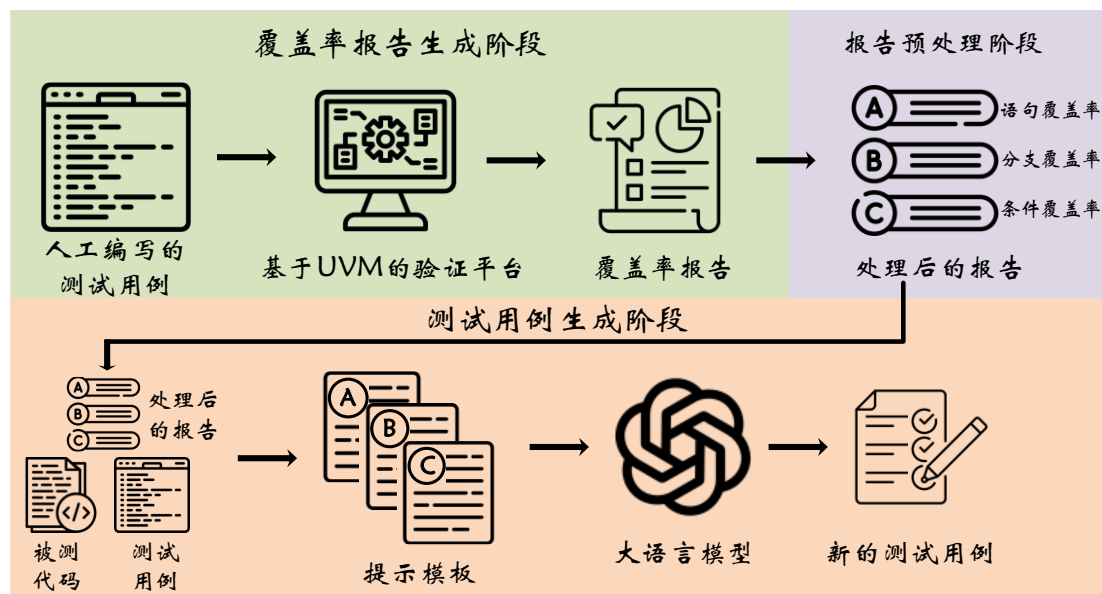


图 3-1 方法流程图

在覆盖率报告生成阶段，首先将人工设计的测试用例输入到搭建好的 UVM 验证平台中，平台根据测试用例执行情况生成覆盖率报告。该报告以网页形式输出，详细列出了各个测试用例对设计代码的覆盖情况。在这个阶段，验证平台通过 UVM 的覆盖率分析机制，生成了多种类型的覆盖率报告，包括语句覆盖率、分支覆盖率和条件覆盖率。每个测试用例所对应的代码覆盖情况都被清晰地呈现在报告中，这为下一步的优化工作提供了宝贵的参考数据。

接下来的报告预处理阶段，主要将上一阶段获得的 Html 格式覆盖率报告转换为 Json 格式。这一转换过程不仅便于后续的数据处理，还能将覆盖率报告的内容进行进一步分类，以便为后续的模型输入提供结构化的格式。转换后的 Json 格式报告会被分类为语句覆盖、分支覆盖和条件覆盖三种类型，这为后续的覆盖率优化和测试用例生成提供了有力支持。

在测试用例生成阶段，本章方法设计了三种提示模板，针对不同类型的覆盖率进行优化，引导大语言模型生成新的测试用例。通过结合覆盖率报告中获得的数据以及被测设计的代码结构，大语言模型能够自动生成更具针对性的测

试用例，旨在提高代码的覆盖率和验证效率。在此过程中，由于大语言模型的生成结果具有随机性和一定的多样性，生成的测试用例可能并不完全满足覆盖率的需求，因此本章方法会根据测试结果进行多次优化。每次优化后，都会通过反馈机制再次输入到模型中，直到覆盖率进一步提升，达到预期的验证效果。

3.3 方法关键步骤

3.3.1 覆盖率报告生成

在验证过程中，生成高质量的覆盖率报告是保证验证效果的前提。在本研究中，覆盖率报告的生成依赖于验证平台。测试用例在验证平台中的执行会生成详细的覆盖率数据，覆盖情况包括语句覆盖、分支覆盖、条件覆盖。每种覆盖标准的生成都对应着不同类型的验证目标。例如，语句覆盖率关注的是设计中每行代码的执行情况，而分支覆盖率关注的是条件语句中的每个分支是否被执行。条件覆盖率则聚焦于验证设计中所有可能的条件组合，以确保没有遗漏。

验证平台基于 SystemVerilog 语言，通过其内部的类库和机制，能够有效地捕获这些覆盖率数据，并输出成网页格式的报告，如图 3-2 所示。

Coverage Summary By Instance:

Scope ▾	TOTAL ▾	Statement ▾	Branch ▾	Condition ▾
TOTAL	77.49	85.67	81.50	65.29
ARINC429_v1_0_S_AXI_inst	59.32	76.35	62.14	39.47
register_trig	77.09	80.00	84.61	66.66
top429_label	82.17	88.18	86.88	71.44

Local Instance Coverage Details:

Total Coverage:					59.32%	59.32%
Coverage Type ▾	Bins ▾	Hits ▾	Misses ▾	Weight ▾	% Hit ▾	Coverage ▾
Statements	1886	1440	446	1	76.35%	76.35%
Branches	1173	729	444	1	62.14%	62.14%
Conditions	342	135	207	1	39.47%	39.47%

Recursive Hierarchical Coverage Details:

Total Coverage:					77.49%	77.49%
Coverage Type ▾	Bins ▾	Hits ▾	Misses ▾	Weight ▾	% Hit ▾	Coverage ▾
Statements	10190	8730	1460	1	85.67%	85.67%
Branches	5613	4575	1038	1	81.50%	81.50%
Conditions	1870	1221	649	1	65.29%	65.29%

图 3-2 覆盖率报告

这些报告不仅包括了每个测试用例的执行结果，还详细列出了代码中哪些部分被覆盖，哪些部分未被覆盖，如图 3-3 所示。

Show All	Show Covered	Show Missing
----------	--------------	--------------

Statement	Hits	Coverage
C:/Users/Administrator/Desktopquesta/../../src/trig.v:19	5555	Covered
C:/Users/Administrator/Desktopquesta/../../src/trig.v:22	140	Covered
C:/Users/Administrator/Desktopquesta/../../src/trig.v:29	1125	Covered
C:/Users/Administrator/Desktopquesta/../../src/trig.v:31	0	ZERO
C:/Users/Administrator/Desktopquesta/../../src/trig.v:37	1125	Covered
C:/Users/Administrator/Desktopquesta/../../src/trig.v:39	1055	Covered
C:/Users/Administrator/Desktopquesta/../../src/trig.v:43	1055	Covered
C:/Users/Administrator/Desktopquesta/../../src/trig.v:46	1055	Covered
C:/Users/Administrator/Desktopquesta/../../src/trig.v:48	0	ZERO
C:/Users/Administrator/Desktopquesta/../../src/trig.v:53	4430	Covered

图 3-3 测试用例执行情况

对于验证平台而言，输出精确的覆盖率报告是确保验证过程高效进行的前提，而该报告则成为了后续测试用例生成的依据。

在本研究中，特别关注的是如何利用这些覆盖率数据进行有效的反馈，从而通过大语言模型生成更精准的测试用例，全面提升覆盖率。

3.3.2 覆盖率报告预处理

由于验证平台生成的覆盖率报告通常以 Html 格式输出，而该格式不易于后续处理，因此本研究提出将报告转换为 Json 格式。通过这一转换，本章方法能够更加方便地对报告中的数据进行结构化处理，使其能够高效地输入到大语言模型中进行后续分析。

在报告的转换过程中，首先对 Html 报告进行解析，提取出各类覆盖率数据，包括语句覆盖率、分支覆盖率、条件覆盖率。接着，将这些数据转化为 Json 格式，以便后续的数据处理和模型输入。Json 格式的请求包含以下主要字段：

- **coverage_type**: 覆盖率类型，如语句覆盖率、分支覆盖率、条件覆盖率

等;

- `file_name`: 被验证的代码文件名;
- `coverage_status`: 对应代码文件中各项覆盖率的执行情况;

Json 格式的优点在于其结构化、易于操作和扩展性强,可以方便地为不同阶段的数据处理提供统一的格式支持。

此外,在预处理阶段,还会根据覆盖率数据的类型将其进行分类,分别提取语句覆盖率、分支覆盖率和条件覆盖率的相关信息。每一类覆盖率数据会被单独处理,以便针对性地生成测试用例。例如,语句覆盖率数据主要关注设计中每行代码的执行情况,而分支覆盖率数据则主要关注代码中条件语句的执行路径,条件覆盖率数据则聚焦于设计中的所有条件组合。通过这种方式,能够更加精确地控制后续测试用例生成的方向,并确保覆盖率的优化更加全面。

3.3.3 测试用例生成

传统的测试用例生成方法通常依赖人工编写,存在效率低下和容易遗漏潜在盲点的问题。为了解决这一问题,本研究提出了一种新的生成策略,结合覆盖率分析与大语言模型的生成能力,从而实现对验证目标的全面覆盖。

覆盖率报告能够反映不同覆盖标准下的验证情况,包括语句覆盖率、分支覆盖率和条件覆盖率。每一种覆盖标准对应着不同的验证目标,因此在生成测试用例时需要采取不同的策略。在这一过程中,本研究的创新之处在于设计了针对各类覆盖标准的提示模板,这些模板能够引导大语言模型生成能够有效覆盖验证目标的测试序列。

对于语句覆盖率,提示模板关注于设计中的具体代码行,要求生成的测试用例能够覆盖到这些关键代码路径,从而确保这些路径得到充分验证,如图 3-4 所示。

Prompt Template^A:

- Your role is an expert in chip validation, particularly in protocol validation. You are highly skilled at analyzing the code under test, test cases, and coverage report, and providing new validation sequences that cover any uncovered code paths, especially the sidelined code. The new validation sequences should aim to cover these uncovered paths as thoroughly as possible, ensuring a more comprehensive test coverage.

- Below are the:

<code under test>: The portion of the code that is being validated.

<test cases>: A set of existing test cases that have been applied to the code, showing the scenarios tested.

<coverage report>: A report indicating which code paths were covered and which were not.

- Your task is to analyze the coverage report and generate new validation sequences to cover the uncovered code paths. Your goal is to improve the validation of the sidelined code and maximize the statement coverage.

- Please respond in the format of:

<new verification sequences>...</new verification sequences>.

图 3-4 针对语句覆盖率的提示模板

对于分支覆盖率，模板设计则重点关注条件语句的各个分支路径。通过分析设计中所有的条件判断及其可能的分支，构建相应的模板，如图 3-5 所示，指导大语言模型生成能够覆盖这些分支路径的测试用例。

Prompt Template^B:

-Your role is an expert in chip validation, particularly in protocol validation. You are highly skilled at analyzing code under test, test cases, and coverage report, and providing new validation sequences that cover any uncovered branch paths, especially the sidelined code. The new validation sequences should aim to cover these uncovered branch paths as thoroughly as possible, ensuring a more comprehensive branch coverage.

-Below are the:

<code under test>: The portion of the code that is being validated.

<test cases>: A set of existing test cases that have been applied to the code, showing the scenarios tested.

<coverage report>: A report indicating which branch paths were covered and which were not.

- Your task is to analyze the coverage report and generate new validation sequences to cover the uncovered branch paths. Your goal is to improve the validation of the sidelined code and maximize the branch coverage.

-Please respond in the format of:

<new verification sequences>...</new verification sequences>.

图 3-5 针对分支覆盖率的提示模板

对于条件覆盖率，模板设计则专注于验证条件表达式，如图3-6所示，确保生成的测试用例能够涵盖设计中所有可能的条件组合，从而确保条件表达式得到全面的验证。

Prompt Template^C:

- Your role is an expert in chip validation, particularly in protocol validation. You are highly proficient at providing new validation sequences through condition coverage and validation sequences of sidelined code. The new validation sequences should aim to cover all possible combinations of conditions in the given condition expressions as thoroughly as possible, ensuring comprehensive validation of the condition expressions in the design.

- Below are the:

<code under test>: The portion of the code that is being validated.

<test cases>: A set of existing test cases that have been applied to the code, showing the scenarios tested.

<coverage report>: A report indicating which condition combinations were covered and which were not.

- Your task is to analyze the coverage report and generate new validation sequences to cover the uncovered condition combinations. Your goal is to improve the validation of the sidelined code and maximize the condition coverage.

- Please respond in the format of:

<new verification sequences>...</new verification sequences>.

图 3-6 针对条件覆盖率的提示模板

在模板设计完成后，将其输入到大语言模型中，模型根据模板生成相应的测试用例。这些测试用例不仅能够有效覆盖特定的验证目标，还具备较高的质量和针对性。然而，由于验证任务的复杂性以及大语言模型生成结果的随机性，初次生成的测试用例可能未能完全达到预期的覆盖效果。因此，将这些生成的测试用例输入到验证平台中进行执行，并通过新的覆盖率报告来评估其效果。

根据新的覆盖率报告，可以识别出仍未覆盖的部分，并将这些未覆盖的部分反馈到验证平台中进行进一步调整。这一过程可以反复进行，通过持续反馈和调整，逐步优化生成的测试用例，最终实现更高的验证覆盖率和更高效的验证过程。

在实际应用中，这一策略显著提高了测试用例的质量和验证效率。通过自

动化的反馈机制，能够不断优化测试用例的覆盖效果，减少人工干预，并缩短验证周期。此外，这一基于大语言模型的测试用例生成策略为面对复杂设计和验证任务时提供了灵活且高效的解决方案，确保了验证过程的全面性和准确性。

3.4 实验设计

本章方法的优化策略主要体现在提升测试用例的质量。首先，基于大语言模型的自动化测试用例生成策略能够根据预定义的提示模板生成多样化、针对性强的测试用例，确保覆盖设计中各个关键模块。

其次，大语言模型的生成能力能够根据历史验证数据不断优化测试用例，生成更有效性的测试用例，从而提高验证的全面性和深度。通过这种策略，验证工作能够在较短的时间内完成更多的验证任务，减少人工干预的需求，并显著提高测试用例的质量。

为了评估这种优化策略的效果，本章设计了相关实验。

3.4.1 实验问题设计

为了对基于大语言模型所生成的测试用例质量展开评估，在本节之中设计了几个关键的实验问题，主要是针对测试用例的覆盖率、质量、多样性以及验证效率这几个方面来做深入的分析，借助这些实验问题，可对大语言模型在提升验证效果时的实际应用效果做出全面评估，具体的实验问题包含以下这些：

（一）大模型生成的测试用例是否提升了覆盖率？

通过评估大语言模型生成的测试用例在语句、分支和条件等覆盖率上的提升情况，探讨大语言模型生成的测试用例在覆盖率上的表现。

（二）大模型生成的测试用例是否具有多样性？

借助对不同测试用例相互间差异展开分析，针对验证路径、输入数据以及条件组合方面出现的变化情况，以此来评估大语言模型所生成的测试用例于覆盖不同测试场景之际的多样性表现，避免出现冗余现象以及存在盲点。

（三）大语言模型生成的测试用例是否覆盖了协议的边界条件？

在本次实验过程当中，着重对协议里的几种典型边界条件给予了关注，像

数据溢出、格式错误、时序错误以及校验和错误等情况，并且针对大语言模型能否有效地覆盖这些边界条件展开了探讨，以此来保证协议在极限状况下的正确性。

（四）相较于传统的基于随机约束生成测试用例的方法，大模型生成的测试用例是否能在相同的验证目标下，提升覆盖率和减少验证时间？

评估大语言模型生成的测试用例，看其在保证覆盖率或者提高覆盖率的情况下，能否减少测试执行所需时间，以此来提升验证工作的效率。

（五）不同的提示模板对生成结果的影响如何？

借助设计多种各异的提示模板，剖析提示模板于生成精准测试用例以及提升验证覆盖率方面所发挥的作用。

3.4.2 实验过程

为全面评测大语言模型于测试用例生成中的成效，本文设计了下述实验流程，保证可针对每个实验问题展开细致分析与验证。

借助基于大语言模型所生成的测试用例来开展语句、分支以及条件覆盖率的分析工作，评估大语言模型生成的测试用例于覆盖率方面的表现情况，这会直接对实验问题（一）给予回应，依靠对比生成的测试用例在不同覆盖率维度上呈现出的表现，可明确地呈现出大语言模型在提升覆盖率方面所发挥的作用。

为剖析测试用例的多样性情况，本章实验对不同生成测试用例之间的差别展开对比，测试路径、输入数据以及条件组合方面的变化，借助量化测试用例间的差异性，来评估所生成的测试用例是否有充足的多样性，并且能否覆盖更多不同场景，避免出现冗余，此过程会帮助回答实验问题（二）。

本章实验评估了大语言模型生成的测试用例在协议的边界条件覆盖能力方面之后对大语言模型所生成的测试用例在协议边界条件覆盖能力上的表现展开了评估，此次实验会着重剖析像数据溢出、格式错误、时序错误以及校验和错误等这类典型的边界条件，依靠把大语言模型生成的测试用例和传统的基于随机约束生成的测试用例进行对比，本章实验会去分析大语言模型能不能有效地生成测试用例来覆盖这些异常状况，以此保证协议在极限条件下的正确性。这可帮助回答实验问题（三）。

为验证大语言模型所生成的测试用例和传统基于随机约束生成的测试用例

谁优谁劣，于相同验证目标下，对验证时间与覆盖率的变化情况展开了对比，本章实验剖析大语言模型生成的测试用例在提升覆盖率之际能否减少验证时间，以此提高验证效率，此部分可帮助解答实验问题（四）。

为剖析不同提示模板对于生成结果所产生的影响，本章实验精心设计并运用了几种各异的提示模板，借助对比不同模板之下生成的测试用例，剖析它们于覆盖率方面存在的差异，这可帮助解答实验问题。（五）。

经过这一系列的实验，本章实验对大语言模型在提高测试用例质量、验证效率以及多样性等方面所有的优势展开了全面评估，同时为后续对验证策略进行优化，提供了有效的实验数据以及分析方面的支持。

3.5 实验结果及分析

3.5.1 实验问题一

表 3-1 显示了通过与大模型的多轮对话后，语句、分支、条件覆盖率的提升情况。

表 3-1 大模型对覆盖率的提升效果

轮数	语句覆盖率	分支覆盖率	条件覆盖率
0	68.45%	60.32%	54.76%
1	81.27%	75.58%	68.19%
2	85.63%	78.91%	72.45%
3	88.92%	81.46%	75.89%
4	90.21%	84.33%	78.53%
5	91.78%	86.77%	80.92%

从表 3-1 中可以看到，在与大语言模型进行首次对话之后，三种覆盖率均有明显提升。展开来说，语句覆盖率从 68.45% 提高到 81.27%，增长了 12.82%；分支覆盖率从 60.32% 提高到 75.58%，增长了 15.26%；条件覆盖率从 54.76% 提高到 68.19%，增长了 13.43%。这些数据说明，借助大语言模型的测试用例生成策略，可有效扩大测试用例的覆盖范围并提高针对性，依靠和大语言模型交互，测试用例逐渐覆盖了设计里更多的功能路径、分支以及条件，提升了验证的全面程度。

在交互轮次不断增加的过程中，从第 1 轮一直到第 5 轮，虽然每一轮的覆盖率提升幅度慢慢变小，不过三项覆盖率都持续呈现出提高的态势。举例来说，

语句覆盖率从原本的 81.27%提升到了 91.78%，提升的幅度达到了 10.51%；分支覆盖率从 75.58%提升至 86.77%，提升了 11.19%；条件覆盖率从 68.19%提升至 80.92%，提升了 12.73%。这些结果说明，即便随着交互轮次增多，提升幅度渐渐趋于平稳，然而大语言模型的生成能力在多个迭代进程里逐步优化了测试用例的质量，让最终生成的测试用例可覆盖更多的设计场景以及功能，使得验证工作逐渐靠近其理想的覆盖效果。

随着测试用例覆盖范围慢慢趋于完善以及对初期覆盖情况进行优化之后，后续轮次可改进的空间就变得比较小。第 5 轮之后覆盖率提升变得缓慢，主要原因在于设计代码里面存在预留功能，也就是说有些功能已经被设计好了并且预留下来以便后续开发使用，不过在当前测试阶段这些功能并没有实际启用或者完全实现。这种“保留性设计”使得某些功能路径、分支以及条件在当前的测试用例中没有被激活，无法让覆盖率得到提升。随着大语言模型生成的测试用例覆盖了大部分已激活的功能，后续提升的幅度就逐渐变小。虽说大语言模型依靠迭代对测试用例进行了优化，然而因为代码中的预留功能还没有启用，没能让覆盖率提高。

借助反馈机制，大语言模型可依据先前测试结果与覆盖率报告，对所生成的测试用例给予动态优化。刚开始生成的测试用例大多时候会出现覆盖不全面或者存在冗余的状况，然而随着轮次不断增加，模型针对测试用例的调整逐渐优化了覆盖范围，提高了测试用例的质量。此过程呈现出大语言模型在优化验证效率及质量方面有很大潜力，还切实减少了人工干预的必要。

3.5.2 实验问题二

在实验问题二中，所关注的重点是大语言模型生成的测试用例是否具有多样性的特征。这里所说的多样性具体指的是，测试用例在覆盖路径、输入数据以及条件组合等方面所呈现出的差异性情况。为了可清晰地揭示出这一特性，本实验采用计算路径差异度的方式来对生成序列的多样性程度进行评估。

路径差异度所体现的是不同测试用例之间于测试路径方面的差异程度。当差异度处于较高水平时，这意味着不同的测试序列涉及了设计里更多的路径以及条件组合，使得验证效果呈现出更为全面的态势。在进行多样性评估期间，先是借助 Jaccard 相似度来对不同测试用例的路径相似度给予衡量，随后运用如

下公式来计算整体的路径差异度：

$$Path\ Diversity = 1 - \frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j=i+1}^N Jaccard\ Similarity(P_i, P_j) \quad (3-1)$$

其中：N是测试用例的总数， P_i 和 P_j 是第i个和第j个测试用例的路径合集， $Jaccard\ Similarity(P_i, P_j)$ 是第i个和第j个测试用例的路径合集的 Jaccard 相似度，计算公式为：

$$Jaccard\ Similarity(P_i, P_j) = \frac{|P_i \cap P_j|}{|P_i \cup P_j|} \quad (3-2)$$

通过上述公式，可以量化测试用例的路径差异度，并在实验中对比较大模型生成的测试用例的路径多样性，实验结果如表 3-2 所示。

表 3-2 大模型生成的测试用例的多样性

轮数	路径相似度
1	0.41
2	0.45
3	0.48
4	0.50
5	0.53

从表 3-2 中可以看到，随着与大语言模型的多轮交互，路径差异度呈现出逐渐增大的趋势，生成的测试用例逐渐涵盖了更多不同的路径和条件组合，从而提高了测试用例的多样性。

3.5.3 实验问题三

在实验问题三中，重点关注大语言模型所生成的测试用例能否有效覆盖协议中的边界条件，特别是异常场景和极限值条件。此次实验的主要目标，在于评估大语言模型生成的测试用例，针对数据溢出，格式错误，时序错误和校验和错误等边界条件的覆盖能力。借助这一分析，可明确大语言模型是否可以有效地覆盖协议中的边界情况。保障协议在极限条件下的正确性。

为了对这一点加以验证，在和大语言模型展开多轮交互的进程当中，着重留意了协议里有可能出现的异常状况。每一轮同模型完成交互之后，所生成的测试用例都会被分析，看其是否覆盖了这些边界条件，实验设计覆盖几个关键点：

数据溢出：测试用例是否能够覆盖协议在数据大小达到极限时的行为，例

如缓冲区溢出或数据传输错误等。

格式错误：测试用例是否能够验证协议格式错误的处理，如无效的数据格式或不符合协议要求的数据结构。

时序错误：测试用例是否能够验证协议在不正确的时间顺序下的行为，例如超时错误或无效的时序执行。

校验和错误：测试用例是否能够覆盖协议在校验和错误情况下的表现，例如数据传输过程中产生的校验错误。

实验结果如表 3-3 所示，列出了每一轮交互中大语言模型生成的测试用例在覆盖这些边界条件方面的表现：

表 3-3 大语言模型生成的测试用例对边界条件的覆盖情况

轮数	数据溢出	格式错误	时序错误	校验和错误
1	否	否	否	否
2	否	否	否	是
3	是	否	否	是
4	是	是	是	是
5	是	是	是	是

观察表 3-3 可发现，随着与大语言模型进行多轮交互，所生成的测试用例逐渐囊括了协议里更多的边界条件。在首次交互之后，生成的测试用例成功覆盖了校验和情况，不过在格式错误、时序错误以及校验和错误这些方面依旧存在不足。随着交互次数的增多，模型对测试用例进行了逐步优化，最终在第四轮的时候成功覆盖了全部四类边界条件。这样的结果说明，大语言模型可对生成的测试用例进行逐步优化，以此保证测试的全面性以及对异常条件的覆盖，验证协议在极限情况下的行为表现。

实验所得到的结果显示，大语言模型借助多轮交互的方式对生成的测试用例进行了优化，在协议的边界条件这一方面，它可有效地覆盖如数据溢出、格式错误、时序错误以及校验和错误等多种异常场景。此项发现证实了大语言模型于提升测试用例质量以及覆盖面这两方面所有的有效性，同时也为后续展开的测试优化以及自动化工作提供了相应的支持

3.5.4 实验问题四

本实验主要是对大语言模型生成的测试用例展开评估，观察其在保证覆盖率或者提高覆盖率的情况下，能否减少测试执行的总体时间，提升验证工作的

效率。展开来说，针对基于随机约束生成的测试用例以及大语言模型生成的测试用例，在达成相同验证目标时的总验证时间进行了比较，同时还分析了这两种方法在覆盖率方面存在的差异。

为实现对比，本实验选定 ARINC429 协议里的数据发送模块当作验证目标。实验过程中，基于随机约束生成的测试用例以及大语言模型生成的测试用例，各自覆盖了该模块的 10 个测试场景，每个测试用例都针对不一样的功能路径、条件以及边界情况。本实验分别记录了这两种方法下生成的测试用例的语句覆盖率、分支覆盖率、条件覆盖率以及总验证时间。

表 3-4 随机约束与大语言模型生成测试用例在覆盖率和验证时间上的对比

方法	语句覆盖率	分支覆盖率	条件覆盖率	总验证时间
基于随机约束生成测试用例	83.45%	77.32%	72.87%	4h
大语言模型生成测试用例	91.78%	86.77%	80.92%	1.3h

从表 3-4 中可以看到，基于随机约束所生成的测试用例跟大语言模型生成的测试用例在覆盖率方面存在着差异。大语言模型生成的测试用例在语句覆盖率、分支覆盖率以及条件覆盖率上都超过了基于随机约束生成的测试用例。展开来说，语句覆盖率从 83.45% 提升到 91.78%，提高了 8.33 个百分点；分支覆盖率从 77.32% 提升到 86.77%，提高了 9.45 个百分点；条件覆盖率从 72.87% 提升到 80.92%，提高了 8.05 个百分点。这些结果说明，大语言模型生成的测试用例可以覆盖更多的功能路径与条件，验证的全面性有了一定程度的提高。

大语言模型生成的测试用例在覆盖率方面表现出色，且验证时间大幅减少。基于随机约束生成的测试用例的总验证时间是 4 小时，大语言模型生成的测试用例只需 1.3 小时，节省了近 70% 的验证时间。这一结果凸显大语言模型在自动化测试方面的优势，能在短时间内生成更多测试用例，提高了测试效率。与基于随机约束生成的测试用例相比，大语言模型生成的测试用例减少了测试时间，在相同时间内达到了更高的测试覆盖率。

实验最终的结果有力证实了大语言模型生成测试用例有的优越性，其在覆盖率方面有明显提高，可保障测试更加全面且有效，还可以大幅缩减验证的整体时间，提升验证工作的效率。这种特性对于那些需要开展大量测试的复杂系统而言非常适用，在处于快速迭代以及追求高效验证的开发环境里，大语言模型的运用将会极大程度减轻人工负担，节省验证资源，提高测试的质量以及速度。

3.5.5 实验问题五

图 3-7 和图 3-8 展示了两种不同的提示模板。图 3-7 为基本的提示模板 1，要求大语言模型根据覆盖列表和验证序列生成新的验证序列以提高覆盖率。图 3-8 为提示模板 2，在模板 1 的基础上，进一步为大语言模型赋予了角色指令，指定模型为芯片验证专家，尤其是在协议验证方面具有丰富经验。模板要求模型根据覆盖列表和验证序列生成新的验证序列。

Prompt Template¹:

-You need to generate new verification sequences to improve coverage based on the coverage list and verification sequences. Below are the <Coverage List>...</Coverage List> and <Verification Sequences>...</Verification Sequences>. Please respond in the format of <New Verification Sequences>...</New Verification Sequences>.

图 3-7 提示模板 1

Prompt Template²:

-Your role is an expert in chip verification, especially in protocol verification. You are highly skilled at generating new verification sequences to improve coverage through coverage analysis and verification sequences. Below are the <Coverage List>...</Coverage List> and <Verification Sequences>...</Verification Sequences>. Please respond in the format of <New Verification Sequences>...</New Verification Sequences>.

图 3-8 提示模板 2

3.3.3 章节中提出的模板在提示模板 2 的基础上做出了进一步的修改和优化。该模板在原有角色指令的基础上，增加了针对不同验证目标（如语句覆盖、分支覆盖和条件覆盖）的细化要求。模板不仅要求模型生成新的验证序列，还要求根据实际的覆盖报告反馈，对生成的测试用例进行自适应调整。这一修改使得验证过程更加精细和高效，能够有效覆盖更多的功能路径和条件组合，从而提高验证的全面性和准确性。

为了比较不同提示模板的效果，本实验采用了与大语言模型的多轮交互方法，每种模板下都与大语言模型进行了 5 轮交互。在每轮交互后，收集生成的

测试用例，并进行语句、分支、条件覆盖率的计算。实验数据包括每种模板下生成的测试用例的覆盖率变化，具体情况如表 3-5 所示。

表 3-5 不同提示模板生成的测试用例的覆盖率变化

轮数	语句覆盖率			分支覆盖率			条件覆盖率		
	PT1	PT2	PT ^A	PT1	PT2	PT ^B	PT1	PT2	PT ^C
1	62.34%	68.45%	75.12%	54.23%	61.54%	70.24%	50.12%	57.64%	65.43%
2	68.92%	74.23%	82.76%	60.76%	67.45%	78.12%	56.34%	64.23%	73.21%
3	74.56%	80.11%	88.34%	67.23%	73.12%	84.54%	61.54%	70.12%	79.65%
4	77.23%	83.45%	91.23%	70.12%	76.54%	87.76%	64.32%	73.54%	82.98%
5	78.45%	85.76%	92.78%	72.32%	79.45%	89.12%	66.21%	76.23%	85.67%

实验结果表明，3.3.3 章节提出的模板在覆盖率提升方面显著优于提示模板 1 和提示模板 2，表现出更高的测试用例质量和全面性。

3.6 本章小结

本章节介绍了基于大语言模型的测试用例生成方法，该方法通过设计代码和覆盖率报告等信息结合提示模板，运用大语言模型优化了验证过程中的测试用例生成策略。通过实验评估，结果表明，大语言模型生成的测试用例在语句覆盖率、分支覆盖率和条件覆盖率方面表现优异。同时，使用大语言模型生成的测试用例显著减少了验证所需的总时间，相较于基于随机约束生成的测试用例，验证效率得到了提升。大语言模型还能生成更多样化的测试用例，提升了测试的全面性。

第 4 章 基于 UVM 的验证平台设计与实现

本文第三章提出大语言模型的测试用例生成方法，以此提高测试用例生成的效率以及覆盖率。该方法借助大语言模型的自动化能力，对验证过程里的用例生成给予优化。基于此验证策略，第四章会详细说明怎样把这一方法运用到 ARINC429 模块的验证平台实现上，本章会呈现如何在 UVM 验证框架下搭建一个模块化的验证平台，并且把大语言模型生成的测试用例融入验证流程，提升验证的全面程度与准确程度。

4.1 ARINC429 模块设计

ARINC429 模块的设计可分为四个主要子模块：发送模块、接收模块、读写控制模块和寄存器组，如图 4-1 所示。每个模块承担着不同的任务，并相互配合实现完整的信号传输与控制功能。接下来，将逐一介绍这四个子模块的设计与实现。

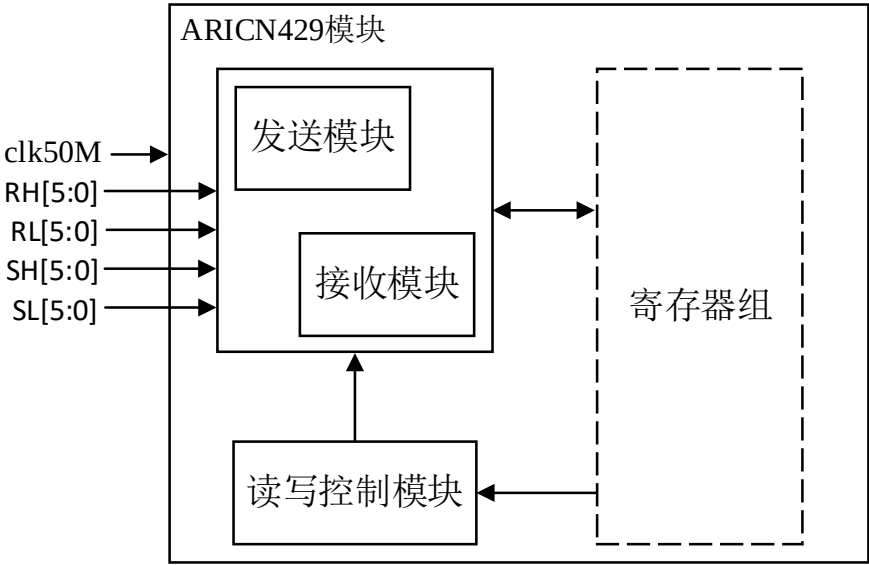


图 4-1 ARINC429 模块架构示意图

4.1.1 发送模块设计

发送模块是 ARINC429 模块中核心的子模块之一，发送模块的结构图如图 4-2 所示，该发送模块对 fifo429 模块、ratecounter 模块、counter32 模块以及 counter 模块进行了例化。发送模块的任务是依据系统配置，借助指定通道发送契合 ARINC429 协议的数据。数据发送配置有灵活性，可选择不同波特率以及决定是否启用奇偶校验，在设计过程中要保证每个发送通道都可以依照协议规定的时序与格式精确发送数据帧，以此保证数据的完整性以及传输可靠性。

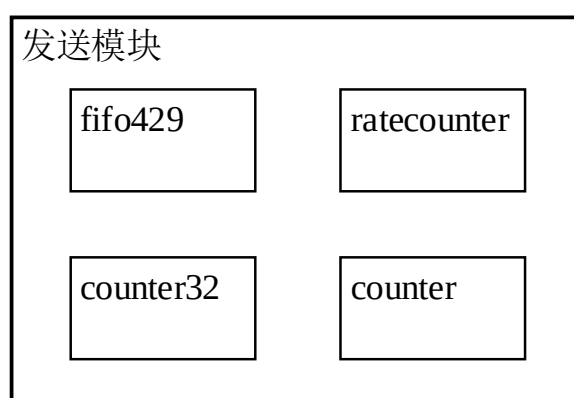


图 4-2 发送模块结构图

在 ARINC429 协议中，发送时序要求十分严格。若 send 模块的发送速率被配置为 100Kbps，则单 bit 时间应保持在 10000ns，并且单 bit 中的高电平和低电平保持时间各为 5000ns，同时 429 数据帧的间隔不应少于 4 个 bit 时间。而若发送速率配置为 12.5Kbps 时，单 bit 时间应为 80000ns，且高、低电平的保持时间均为 40000ns，数据帧间隔仍不小于 4 个 bit 时间。为了实现这些时序要求，发送模块设计依赖于四个核心子模块的配合与协调。

首先，fifo429 模块负责数据的存储与缓存。每一路发送通道都配备了 1024 个数据字的 FIFO 缓存，数据将按照写入顺序进行发送。当 FIFO 缓存满时，新写入的数据将会被丢弃。通过 FIFO 缓存的机制，发送模块能够确保数据的顺序性和完整性，避免数据丢失。

其次，ratecounter 模块在发送模块中承担着波特率计数的关键任务。该模块确保每个比特的时间与预定的波特率匹配，从而精确控制发送时序。

第三，counter32 模块负责 32 位数据的计数。在发送数据时，该模块会根据配置的波特率调整数据位的发送速率，确保数据的时序精度，符合协议的要求。

该模块在确保每个数据位的准确发送的同时，协调各个比特位的计时与控制。

最后，**counter** 模块主要用于空闲周期的计数。该模块通过精确计时控制数据帧间隔的时序，确保 429 数据帧间隔不小于 4 个 bit 时间。

通过这四个模块的紧密配合，发送模块能够实现对数据发送时序的精确控制，确保按照 ARINC429 协议的要求进行数据传输。这些模块共同协作，确保数据的正确发送，并且使发送模块能够适应不同的波特率和校验配置，满足项目中的实际需求。

4.1.2 接收模块设计

接收模块的结构图如图 4-3 所示，接收模块例化了 **fifo429** 模块、**ratecounter** 模块、**ratecounter2** 模块、**counter32** 模块和 **counter** 模块。接收模块根据选择的通道和接收的配置完成相应通道的数据接收工作，根据不同的配置，接收的波特率可以是 12.5Kbps 或 100Kbps，数据接收可配置为奇偶校验或无校验。

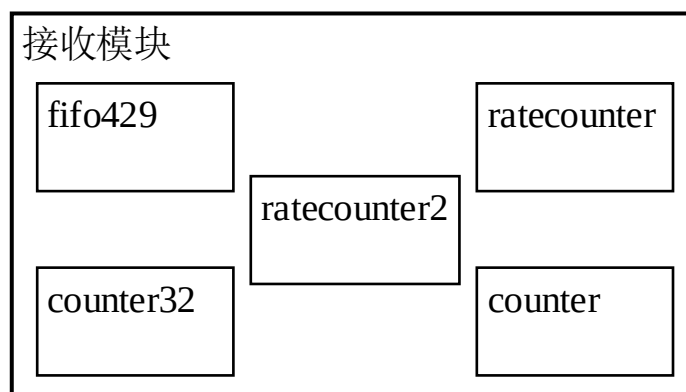


图 4-3 接收模块结构图

具体而言，**fifo429** 模块负责缓存接收到的数据，并按照接收顺序进行存储。每个接收通道都有独立的 FIFO 缓冲区，确保接收到的数据能够顺序存放并传输，避免数据丢失或冲突。

ratecounter 模块则用于控制接收数据的波特率计数，确保每个 bit 的时间符合协议要求。当接收速率设置为 100Kbps 时，单 bit 低电平的最短保持时间为 2720ns，下降沿的建立时间和保持时间不小于 4 个时钟周期；而 12.5Kbps 速率下，单 bit 低电平的最短保持时间为 23760ns，确保协议中的时序要求得到满足。

ratecounter2 模块通过对接收到的数据打八拍的方式，要求前四拍相等、后四拍相等，并且保证第四拍与第五拍的状态相反。此时序控制保证了接收数据

在协议规定的时序条件下被正确接收。

counter32 模块则用于管理 32 位数据计数，在接收过程中，确保每一个数据位都能够按照协议要求的时序精确接收。

counter 模块则用于控制接收数据帧间的时间间隔，确保单位 bit 之间的最大时间间隔为 4 个 bit，保证数据的稳定传输和接收时序的准确性。

这些模块紧密配合，确保了接收数据的正确性和时序的精确控制，从而使得接收模块能够在多种配置下稳定地工作，满足 ARINC429 协议的接收要求。

4.1.3 读写控制模块设计

读写控制模块在 ARINC429 协议的实现里占据着关键地位，它承担着管理数据读写行为的职责，目的在于保证接收以及发送模块可以准确无误地处理数据传输工作。该模块的主要功能是借助接收寄存器组当中的值，产生相应的触发信号，这些信号对接收 FIFO 和发送 FIFO 的使能操作起到控制作用，最终达成数据的顺利读取与写入。

展开来说，读写控制模块最先接收源自寄存器组的数据，紧接着依据信号处理逻辑产生触发信号。这些触发信号会对接收 FIFO 以及发送 FIFO 的使能信号给予控制，以此保障数据流可顺利地传输。当接收 FIFO 的使能信号被触发之际，模块会读取接收 FIFO 里的数据，以此保证数据可得到及时处理；而当发送 FIFO 的使能信号被触发之时，模块会把数据写入发送 FIFO，保证数据可依照 ARINC429 协议规范的时序来进行传输。

4.1.4 寄存器组设计

ARINC429 模块构建了一个寄存器组，详情见表 4-1，其作用是对源自 CPU 的读写数据以及配置数据给予缓存与存储。此寄存器组有独立的地址空间，里面包含多个存储寄存器，像 slv_reg0 至 slv_reg4，这些用于存储每条数据的收发状态。该模块还设计了多个配置寄存器，例如 slv_reg5 至 slv_reg8，用来设置数据的波特率、校验方式等相关内容。

存储寄存器部分（slv_reg0 至 slv_reg4）主要用于存储第 1 至第 5 路的数据，这些数据通过接收和发送 FIFO 进行存储和传输。每个存储寄存器都具有独立的

地址，能够独立存储不同的数据流，确保数据的高效读取和写入。具体来说，slv_reg0 至 slv_reg4 分别存储从第 1 路到第 5 路的数据，便于后续的操作和控制。

控制寄存器部分（slv_reg5 至 slv_reg8）则负责模块的配置和控制。slv_reg5 和 slv_reg6 分别用于配置第 1 至第 4 路和第 5 路的发送配置，slv_reg7 和 slv_reg8 则用于配置第 1 至第 4 路和第 5 路的接收配置。通过这些寄存器，用户可以灵活配置数据传输的相关参数，如波特率、校验方式等。

除了这些存储和配置寄存器外，寄存器组还包括多个状态寄存器。slv_reg9 和 slv_reg10 分别用于获取发送缓冲区和接收缓冲区的状态，而 slv_reg11 则用于存储 ARINC429 协议的版本更新时间。其他控制寄存器如 slv_reg12、slv_reg13 和 slv_reg14 分别用于配置 429 内部自环、接收通道使能以及 FIFO 复位。

寄存器组的设计确保了数据的高效存储与访问，支持 CPU 对各个功能模块的灵活控制，为系统的高效运行提供了保障。

表 4-1 寄存器列表

寄存器名	偏移地址	读/写 (R/W)	功能描述
slv_reg0	0x000	R/W	存储第 1 路收发数据
slv_reg1	0x004	R/W	存储第 2 路收发数据
slv_reg2	0x008	R/W	存储第 3 路收发数据
slv_reg3	0x00C	R/W	存储第 4 路收发数据
slv_reg4	0x010	R/W	存储第 5 路收发数据
slv_reg5	0x014	W	配置第 1-4 路发送配置
slv_reg6	0x018	W	配置第 5 路发送配置
slv_reg7	0x01C	W	配置第 1-4 路接收配置
slv_reg8	0x020	W	配置第 5 路接收配置
slv_reg9	0x024	R	发送缓冲状态
slv_reg10	0x028	R	接收缓冲状态
slv_reg11	0x02C	R	存储 429 版本更新时间
slv_reg12	0x030	W	配置 429 内部自环
slv_reg13	0x034	W	配置接收通道使能
slv_reg14	0x038	W	配置 FIFO 复位

4.2 验证平台设计

验证工作的核心在于验证平台的搭建。图 4-4 为本文搭建的基于 UVM 的 ARINC429 模块验证平台，通过构建一个模块化的验证平台，为测试用例的执行提供强有力的支持，提升验证过程的全面性和准确性。

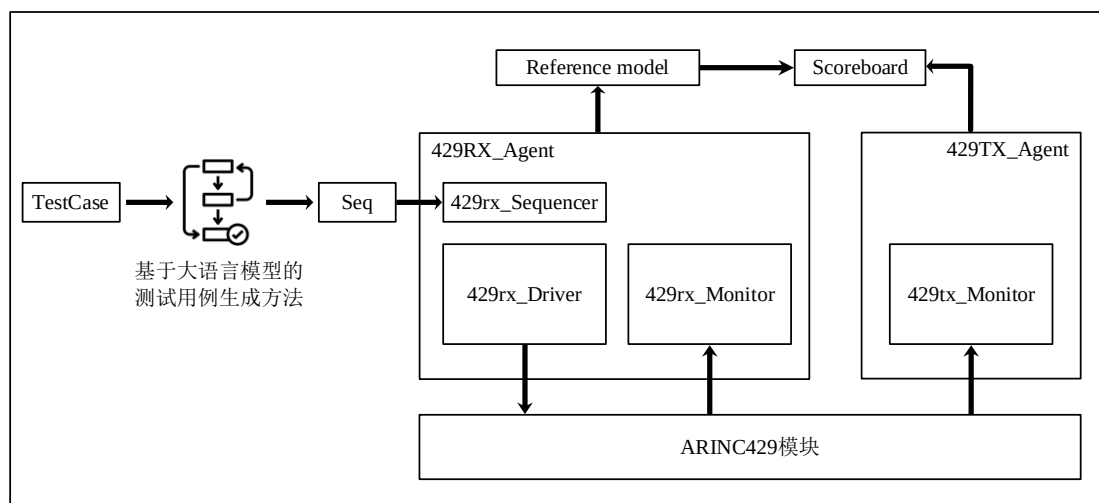


图 4-4 基于 UVM 的 ARINC429 模块验证平台

整个验证平台围绕 ARINC429 模块展开，通过 429RX_Agent 和 429TX_Agent 分别实现输入端和输出端的信号交互。在输入端，429RX_Agent 包含 Sequencer、Driver 和 Monitor，其中 Sequencer 负责生成测试激励，Driver 将激励转化为信号并传递给 ARINC429 模块，Monitor 则捕获 ARINC429 模块的输入信号并提取数据。在输出端，429TX_Agent 的 Monitor 实时监控 ARINC429 模块的输出信号并提取数据用于验证。整个系统还包含一个参考模型（Reference Model），用于生成理想的参考输出，与 Scoreboard 模块结合，将参考数据与 Monitor 提取的实际数据进行比较，以验证 ARINC429 模块的功能正确性。

在测试用例生成方面，首先通过手动生成初步测试用例，确保覆盖基础场景。随后，利用基于大语言模型的测试用例生成方法，进一步优化和扩展测试用例，涵盖更为复杂和边界的测试场景。大语言模型根据现有测试用例和覆盖率报告等信息，生成新的测试场景，自动化地提升了测试用例的丰富性与全面性。这种结合手动和自动生成的策略，不仅提高了测试覆盖率，也加速了测试用例的生成过程。

通过这种模块化的架构设计和结合大语言模型的测试用例生成方法，该验证环境能够有效实现覆盖率驱动的闭环验证，提高 ARINC429 总线协议的验证效率和准确性。

4.3 验证平台实现

在本研究的验证平台设计中，采用了UVM，并围绕ARINC429总线协议的验证需求，构建了包括Driver、Monitor、Sequencer、Agent、Scoreboard和Reference Model在内的核心组件。各个组件的设计和实现紧密围绕着验证任务，确保能够高效、准确地验证目标设计的功能和性能。

4.3.1 Driver 实现

Driver的主要任务是从Sequencer接收生成的测试事务，并将其转化为符合ARINC429总线协议的数据格式，然后传输至被测设计。Driver不仅需要按照协议规定的时序控制数据包的每个位的输出，还需要模拟协议中的正常传输和异常传输，以验证被测模块在各种情况下的功能表现。

在实现上，本文基于UVM的uvm_driver类来构建Driver的功能。Driver通过与被测模块的接口连接，利用虚拟接口完成与模块之间的数据交互。在UVM的run_phase中，Driver会持续不断地从Sequencer中获取测试事务，并根据事务内容生成符合ARINC429协议的数据包。

具体来说，Driver会根据ARINC429协议的要求设置数据包中的各个字段。首先，它会从Sequencer接收到一个32位的数据字，并将其解析为ARINC429协议中的数据帧。Driver会根据协议规定设置起始位、数据位、校验位等内容。

此外，Driver还需要处理协议中的奇偶校验。根据ARINC429协议的要求，数据字中的奇偶校验位用于保证数据的完整性。Driver会根据数据字的内容计算并设置校验位，确保数据在传输过程中不被损坏。

通过上述步骤，Driver能够严格按照ARINC429协议的时序要求，准确地将数据驱动到被测设计中，为后续的验证工作提供有效的测试用例。

部分核心代码如下所示：

```
1. task arinc429_driver::drive_one_pkt(arinc429_transaction tr);
2.     bit parity_bit;
3.     int i;
4.     bit [31:0] data = tr.data429;
5.     int delay_cycles = arinc429_cfg.driver_rate ? 125 : 1000; // 根据当前的速率
    配置，计算延迟
6.     parity_bit = utils::calculate_parity(arinc429_cfg.driver_parity,data); //
    校验位计算
7.     data[31] = parity_bit;
```

```

8.   for (i = 0; i < 32; i++) begin // 发送逻辑，逐位发送数据
9.       vif.SH = data[i];
10.      vif.SL = ~data[i];
11.      repeat (delay_cycles - 1) @(posedge vif.clk);
12.      vif.SH = 1'b0;
13.      vif.SL = 1'b0;
14.      repeat (delay_cycles - 1) @(posedge vif.clk);
15.  end
16.  @(posedge vif.clk);
17.  vif.SH = 1'b1;
18.  vif.SL = 1'b0;
19.  d2a_port.put(tr);
20. endtask

```

通过上述设计，Driver能够有效地模拟ARINC429总线协议中的传输场景，并为验证平台提供稳定的数据输出。

4.3.2 Monitor 实现

Monitor的主要任务是捕获接收到的数据并进行协议一致性检查，确保数据包符合ARINC429协议的传输要求。每当Monitor捕获到一个完整的数据帧时，它需要将数据的各个字段，包括时序信息，提取出来，并将这些信息交给Scoreboard进行进一步的验证和分析。

Monitor的设计基于UVM中的uvm_monitor类，并通过虚拟接口与被测模块的输出端口连接。在run_phase任务中，Monitor会持续监听被测模块的输出信号，捕获数据并生成相应的事务对象。这些事务对象包含了数据包中的所有重要字段，如数据位、标签（Label）、SSM（Sign-Status Matrix）、奇偶校验位等，确保捕获的每一帧数据都是完整且符合协议规定的。

部分核心代码如下所示：

```

1. for (i = 0; i < 32; i++) begin // 循环 32 个周期，捕获数据位
2.     @(posedge vif.clk);
3.     if (vif.SH && ~vif.SL) begin
4.         data_q.push_back(1'b1);
5.     end else if (~vif.SH && vif.SL) begin
6.         data_q.push_back(1'b0);
7.     end else begin

```

```

8.      `uvm_error("arinc429_monitor", "SH and SL are both 1'b1.");
9.      end
10.     repeat (delay_cycles - 1) @(posedge vif.clk);
11. end

```

4.3.3 Sequencer 实现

Sequencer 的主要任务是根据测试用例生成测试事务，并将这些事务发送给 Driver 进行处理。每个事务代表一个完整的协议数据包，通常包含了数据包中的各个字段如数据位、标签、SSM（Sign-Status Matrix）位等。Sequencer 需要确保生成的事务不仅符合协议的基本格式要求，还能覆盖到不同的测试场景，包括正常的数据传输、边界条件、异常数据传输等。

在设计上，Sequencer 通过继承 UVM 中的 `uvm_sequencer` 类来实现，其核心任务是根据测试需求从事务队列中获取事务对象，并将其传递给 Driver 进行执行。每个事务在生成时，都根据协议要求对数据进行格式化，并通过 Sequencer 与 Driver 进行接口连接。

具体来说，Sequencer 通过 `body` 任务循环地从事务队列中取出一个事务对象，每个事务对象表示一个完整的 ARINC429 数据包。它确保事务的生成顺序和时序一致性，即事务按照预定的顺序和时序规则依次传递给 Driver。在整个验证过程中，Sequencer 不仅负责将数据传递给 Driver，还通过控制事务的生成顺序，确保测试覆盖了从正常数据到异常数据的各种场景。

通过上述设计，Sequencer 组件不仅能够生成和传递协议数据包，还能够根据不同的测试需求，灵活地控制事务的生成、顺序和执行时机，从而支持多种不同的测试场景。这使得 Sequencer 在整个验证流程中扮演了至关重要的角色，帮助测试人员全面验证 ARINC429 协议的实现。

核心代码如下所示：

```

1. class arinc429_sequencer extends uvm_sequencer#(arinc429_transaction);
2.     `uvm_component_utils(arinc429_sequencer)
3.     function new(string name, uvm_component parent);
4.         super.new(name, parent);
5.     endfunction : new
6. endclass : arinc429_sequencer

```

Sequencer 的灵活性使得它能够支持多种不同的测试场景，并控制事务的生成顺序。

4.3.4 Agent 实现

本文为发送模块和接收模块分别设计了两个 Agent，每个 Agent 负责其对应模块的验证任务，并独立执行相关功能的验证。通过 Agent 的设计，将验证环境中的各个功能模块进行了模块化，每个模块负责其特定的验证任务，从而大大提高了验证的可扩展性和可维护性。Agent 的设计使得每个功能模块的验证可以独立进行，同时也简化了验证平台的管理和配置工作。

具体来说，发送模块和接收模块的 Agent 分别负责其对应模块的验证任务。发送模块 Agent 主要通过 Driver 组件向 DUT 发送符合 ARINC429 协议的数据，而接收模块 Agent 则通过 Monitor 组件捕获接收的数据并进行协议一致性检查。

每个 Agent 都继承了 UVM 的 `uvm_agent` 类，具备一定的灵活性，以支持在不同的测试场景下执行相应的任务。在验证流程中，Agent 会根据配置的不同进行相应的初始化操作。首先，Agent 会通过其 `build_phase` 阶段进行初始化，配置内部的 Driver、Monitor 和 Sequencer 组件。如果验证需求中需要执行完整的功能测试，则 Agent 将启动 Driver 和 Monitor 两个组件，确保数据的驱动和监控都能按协议要求进行；若只需要进行部分协议的验证，Agent 则可以选择只启动 Monitor 组件，用于检查接收端的数据是否符合预期。通过这种灵活的配置，Agent 能够满足不同测试场景下的需求，确保验证过程的高效性与全面性。

通过对发送模块和接收模块各自设计 Agent，本文有效地将验证任务分离，并且通过模块化设计提升了整个验证平台的可扩展性。这种设计使得测试人员可以方便地根据实际需要修改或扩展特定模块的功能，而不会影响到整个验证环境的稳定性和结构性。

核心代码如下所示：

```
1. class arinc429_agent extends uvm_agent;
2.     arinc429_sequencer sequencer;
3.     arinc429_driver driver;
4.     arinc429_monitor monitor;
5.     function new(string name = "arinc429_agent", uvm_component parent);
6.         super.new(name, parent);
```

```

7.     sequencer = arinc429_sequencer::type_id::create("sequencer", this); //创建并实例化 arinc429_sequencer 组件
8.     driver = arinc429_driver::type_id::create("driver", this); //创建并实例化 arinc429_driver 组件
9.     monitor = arinc429_monitor::type_id::create("monitor", this); //创建并实例化 arinc429_monitor 组件
10.    endfunction
11. endclass

```

通过 Agent 的设计，每个功能模块都可以独立进行验证，验证环境的模块化设计提高了验证的可扩展性和可维护性。

4.3.5 Scoreboard 实现

Scoreboard 通过与 Monitor 连接，获取捕获的事务，并根据预定义的期望结果进行比较。如果发现不一致，Scoreboard 会记录相关的错误信息并提供详细的反馈。Scoreboard 的设计遵循 UVM 的 uvm_scoreboard 类，可以灵活地处理不同类型的数据验证，核心代码如下所示：

```

1. class arinc429_scoreboard extends uvm_scoreboard;
2.     rand bit [31:0] expected_data;
3.     rand bit [31:0] actual_data;
4.     // 记录错误计数
5.     int error_count;
6.     function new(string name = "arinc429_scoreboard", uvm_component parent);
7.         super.new(name, parent);
8.         error_count = 0;
9.     endfunction
10.
11.    task compare_data(bit [31:0] expected, bit [31:0] actual);
12.        // 比较实际数据与期望数据是否一致
13.        if (expected != actual) begin
14.            `uvm_error("arinc429_scoreboard", $sformatf("Data mismatch! Expected: %h, Actual: %h", expected, actual));
15.            error_count = error_count + 1; // 增加错误计数
16.        end else begin
17.            `uvm_info("arinc429_scoreboard", "Data match successful.", UVM_LOW);
18.        end
19.    endtask

```



```
20.  
21. // 在 Scoreboard 完成所有验证后打印错误计数  
22. task report();  
23.     `uvm_info("arinc429_scoreboard", $sformatf("Total errors detected: %0d",  
        error_count), UVM_LOW);  
24. endtask  
25.  
26. endclassendclass
```

通过这种方式, Scoreboard 能够有效地对验证结果进行核对, 并提供错误检测与反馈机制。

4.3.6 Reference Model 实现

Reference Model 的主要任务是根据 ARINC429 总线协议的要求生成正确的数据帧。该模型通常实现为一个简单的生成器, 通过协议的参数生成数据, 并输出到验证环境中的其他组件, 核心代码如下所示:

```
1. class arinc429_reference_model;  
2.     function bit [31:0] generate_data(bit [31:0] input_data);  
3.         bit [31:0] data;  
4.         data = { input_data[31:0]};  
5.         return data;  
6.     endfunction  
7. endclass
```

Reference Model 的实现需要结合协议的具体规范进行调整, 并确保所生成的数据符合协议的所有要求。

4.3.7 测试用例生成实现

在本验证平台中, 大语言模型通过 API 接口与 UVM 验证环境进行集成。在测试用例生成阶段, 首先由人工设计的基础测试用例作为输入, 然后通过调用大语言模型进一步生成多样化的测试用例。这一过程中, Python 脚本作为中介, 负责与大语言模型的交互, 而验证平台的核心代码仍然是基于 SystemVerilog 编写的。

大语言模型生成测试用例的过程大致如下：通过 API 请求将手动编写的初始用例传递给大语言模型，模型根据历史数据和上下文信息生成新的测试激励。生成的测试激励返回给 Python 脚本，再由 Python 脚本将其传递到 SystemVerilog 验证环境中执行。验证平台中相应的 Agent、Driver 等模块处理这些测试激励，并在执行过程中监控和记录相关信号，最终通过 Scoreboard 进行验证。

伪代码流程如下：

```
1. #调用大语言模型生成测试用例
2. def generate_test_cases(initial_cases):
3.     # 发送请求到大语言模型
4.     test_case_data = call_language_model(initial_cases)
5.     # 返回生成的测试用例
6.     return test_case_data
7.
8. #接收并执行测试用例
9. task execute_test_case(input test_case);
10.    // 驱动信号生成与传输
11.    driver.send(test_case)
12.    // 捕获响应信号
13.    monitor.capture_response()
14.    // 执行功能验证
15.    scoreboard.compare(expected, actual);
16. endtask
```

通过这种结合方式，验证平台能够在传统的人工测试用例基础上，通过大语言模型进一步丰富测试场景，提高测试覆盖率和验证的全面性。

4.4 本章小结

本章介绍了基于UVM的ARINC429总线协议验证平台的设计与实现过程。首先，明确了系统的功能需求，重点阐述了发送模块、接收模块、读写控制模块和寄存器组的设计。随后，详细介绍了 Driver、Monitor、Sequencer、Agent、Scoreboard 和 Reference Model 等关键组件的实现，确保了验证过程的全面性与高效性。最后，讨论了如何将大语言模型生成的测试用例与验证平台结合，通

过自动化测试用例生成和反馈机制，显著提高了验证效率和准确性，确保了 ARINC429 协议的全面验证。

第5章 仿真验证与结果分析

在第三章中，本文提出了一种基于大语言模型的测试用例生成方法，利用自动化生成测试用例的方式来提升验证效率，把设计代码、覆盖率报告等信息输入大语言模型，再结合自定义的提示模板，生成有针对性的测试用例，这种方法切实减少了手动编写测试用例的需求，优化了测试覆盖率，还提升了验证的全面程度。

第四章依据第三章所阐述的方法，着重介绍了怎样把这些优化策略运用到实际的 ARINC429 总线协议验证平台的设计以及实现工作当中。借助 UVM 框架，成功构建起一个有模块化特点并且高度自动化的验证环境。这个环境包含了 Driver、Monitor、Sequencer、Agent、Scoreboard 以及 Reference Model 等一系列关键组件。在这个平台里，将大语言模型生成的测试用例与之相结合，切实提高了验证过程的效率以及准确性。

依据第三章与第四章所开展的工作，本章会呈现利用 EDA 工具对 ARINC429 模块实施仿真验证的方式。本章会剖析仿真波形的结果以及覆盖率数据，还会评估大语言模型生成的测试用例在仿真里的表现，借助这些验证，可验证平台的有效性、可靠性，还可为后续的优化给予反馈。

5.1 仿真环境

在平台搭建完成后，需要创建仿真环境，进一步调用验证平台。当前仿真采用主流仿真工具 Questa Sim 进行编译和仿真。Questa Sim 在启动后会读入验证平台及 DUT，先对它们进行编译，转换为可执行文件，进一步根据参数进行仿真，并生成相应的波形。当前仿真采用 Questa Sim 查看波形。

为了实现编译仿真过程的自动化运行，本文还使用了 Makefile 脚本来对编译流程和编译选项进行控制。本文在 Makefile 中使用了三段式编译方法，通过“comp”、“elab”和“run”三个阶段分别实现了分析、例化和仿真三个过程。

Makefile 部分代码如下：

```
1. MODE=gui
2. TEST=IO_TC001
3.
4. ##Make lib_gen should be excuted before any simulation. And should be
   excuted only once.
5. lib_gen: log_dir cp_ini rtl_lib tb_lib tc_lib sim_top sim_opt sim_opt_debug
6.
7. ##
8. tc_test:
9.     tc_lib sim_top sim_opt_debug debug
10.
11. ##run test with waveform.
12. ##e.g.  make all TEST=TC02
13. all:dir work_lib sim_cov_do run_cov
14.
15. ##debug use. Run only with one case. e.g. make debug TEST=TC02
16. debug:dir_debug work_lib_debug sim_cov_do_debug run_debug
17.
18. ##run all testcase with coverage
19. ##If all of the test cases are stable, make run_cov_all can generate all the
   coverage files automaticlly.But not merge ucdb and generate report.
20. run_cov_all:
21.  make all TEST=IO_TC001 MODE=c
22.  make all TEST=IO_TC002 MODE=c
23.  make all TEST=IO_TC003 MODE=c
24.
25. ## merge genarate coverage report and open the report
26. cov_gen: cov_merge cov_report open_cov_report
```

5.2 测试用例

测试用例设计是仿真验证中的核心部分，直接关系到验证平台的有效性和全面性。为了确保 ARINC429 协议模块的每个功能和性能要求都能够被充分测试，本文设计了不同类型的测试用例，这些用例覆盖了协议的基本功能、时序准确性、错误处理能力等。具体而言，本文设计了以下几类测试用例：

5.2.1 数据发送功能测试

数据发送功能是 ARINC429 协议验证中的关键部分，确保每个发送通道能够按照协议要求正确发送数据是验证的基本目标。在本次验证中，测试用例分为两个部分：一部分是手动编写的测试用例，另一部分是基于大语言模型自动生成的测试用例。

首先，本文设计了手动编写的测试用例，核心代码如下所示：

```
1. task 429_TC043::run_phase(uvm_phase phase);
2.   phase.raise_objection(this);
3.   $display("429_TC043 STARTING...");
4.   //第1路到第4路发送通道奇偶校验 波特率
5.   tb_system.pcie_write(32'h0000_1014, 32'h1018_0008);
6.   //第1路发送通道 100kbps 偶校验
7.   tb_system.pcie_write(32'h0000_1000, 32'h2518_4511); //预期 100us 出结果
8.   //第2路发送通道 100kbps 奇校验
9.   tb_system.pcie_write(32'h0000_1004, 32'h8478_9741); //预期 100us 出结果
10.  //第3路发送通道 12.5kbps 偶校验
11.  tb_system.pcie_write(32'h0000_1008, 32'h646b_4da1); //预期 400us 出结果
12.  //第4路发送通道 12.5kbps 奇校验
13.  tb_system.pcie_write(32'h0000_100C, 32'hae4f_4e61); //预期 400us 出结果
14.  #3ms phase.drop_objection(this);
15. endtask
```

主要针对 ARINC429 协议中的数据发送时序和数据帧结构进行验证。接下来，通过大语言模型生成的测试用例进一步扩展了验证范围，大语言模型能够基于先前的验证结果生成优化过的测试用例，模拟不同场景下的数据发送过程，并通过波形分析对比协议要求，确保各个时序部分符合标准。

大语言模型生成的测试用例部分代码如下：

```
1. task 429_TC044::run_phase(uvm_phase phase);
2.   phase.raise_objection(this);
3.   $display("429_TC044 STARTING...");
4.   //第1路到第4路发送通道奇偶校验 波特率
5.   tb_system.pcie_write(32'h0000_1014, 32'h0800_1810);
6.   //第1路发送通道 12.5kbps 奇校验
7.   tb_system.pcie_write(32'h0000_1000, 32'h1324_7563); //预期 100us 出结果
```

```

8.    //第2路发送通道 12.5kbps 偶校验
9.    tb_system.pcie_write(32'h0000_1004, 32'hba39_cef8);    //预期 100us 出结果
10.   //第3路发送通道 100kbps 奇校验
11.   tb_system.pcie_write(32'h0000_1008, 32'h5192_aa55);    //预期 400us 出结果
12.   //第4路发送通道 100kbps 偶校验
13.   tb_system.pcie_write(32'h0000_100C, 32'hf791_1299);    //预期 400us 出结果
14.   #3ms phase.drop_objection(this);
15. endtask

```

5.2.2 数据接收功能测试

数据接收功能是 ARINC429 协议验证中的核心部分，确保接收模块在各种波特率和配置条件下能够正确接收并解析数据是验证的关键目标。在本次验证中，测试用例分为手动编写和基于大语言模型自动生成的两部分。

首先，手动编写的测试用例仅涵盖了特定的配置条件，即 12.5Kbps 波特率与奇校验方式，其验证了接收模块在该配置下能否正确接收到数据，并确保接收到的数据与发送的数据一致。在这一基础测试的过程中，确保接收模块在给定的配置下能满足 ARINC429 协议的要求，且数据解析准确。核心代码如下所示：

```

1. task 429_TC049::run_phase(uvm_phase phase);
2.   automatic logic [31:0] recv_buf_status = 32'b0;
3.   automatic logic [31:0] receive_data = 32'b0;
4.   phase.raise_objection(this);
5.   $display("429_TC049 STARTING...");
6.
7.   //发送通道配置 偶校验 100kbps slv_reg32 前四发送通路配置
8.   tb_system.pcie_write(32'h0000_0014, 32'h1010_1010);
9.   //发送数据 4通路 slv_reg0 - slv_reg3
10.  for (int i = 0; i < 4; i++) begin
11.    tb_system.pcie_write(32'h0000_1000 + i * 4, 32'h1234_5678 + i);
12.  end
13.  //接收通道配置 偶校验 前4路接收通路配置
14.  tb_system.pcie_write(32'h0000_001C, 32'h1010_1010);
15.
16.  //等待前4通路有数据 接收 FIFO 缓冲状态
17.  wait(tb_system.u_top.system_user.system_i.ARINC429_0.inst.ARINC429_v1_0_S_AX
      I_inst.recv_buf_status==24'h0000ff);

```

```

18.
19.  tb_system.pcie_read(32'h0000_0028, 0, recv_buf_status);
20.
21.  for (int i = 0; i < 4; i++) begin
22.      if(recv_buf_status[i]) begin
23.          tb_system.pcie_write(32'h0000_0034, 32'h5500_0000 + i);
24.          tb_system.pcie_read(32'h0000_0000 + i * 4, 0, receive_data);
25.          $display("channel %d is not empty",i);
26.          $display("channel %d receive data is %x", i, receive_data);
27.      end
28.  end
29.  #100us phase.drop_objection(this);
30. endtask

```

大语言模型生成的测试用例则进一步拓宽了验证的范围。通过结合覆盖率报告，模型自动生成了不同波特率（如 12.5Kbps 和 100Kbps）和不同校验方式（如奇校验、偶校验、无校验）的多种测试用例。这些自动生成的用例覆盖了不同的配置条件，确保了接收模块能够在各类不同的设置下进行验证。具体而言，大语言模型根据前述手动测试用例的验证结果（覆盖率数据），优化了后续生成的测试用例。

大语言模型生成的测试用例部分代码如下：

```

1.  //发送通道配置 偶校验 100kbps slv_reg32 前四发送通路配置
2.  tb_system.pcie_write(32'h0000_0014, 32'h0808_0808);
3.  //发送数据 4 通路 slv_reg0 - slv_reg3
4.  for (int i = 0; i < 4; i++) begin
5.      tb_system.pcie_write(32'h0000_1000 + i * 4, 32'h1234_5678 + i);
6.  end
7.  //接收通道配置 偶校验 前 4 路接收通路配置
8.  tb_system.pcie_write(32'h0000_001C, 32'h0808_0808);

```

模型基于先前的反馈动态调整生成的测试用例，进一步优化了接收模块在复杂场景下的表现。通过这种方式，测试用例不仅仅局限于标准配置下的测试场景，还能模拟更多的异常情况，从而提升验证的全面性。

5.2.3 寄存器基础功能测试

寄存器功能测试的目标是验证寄存器的基本读写能力。在手动编写的测试用例中，验证了寄存器在基础数据操作下的功能，主要通过设定特定的寄存器地址，向寄存器写入预设数据，并读取该寄存器的内容。通过对比写入数据与读取数据的一致性，验证了寄存器的基本稳定性和可靠性。核心代码如下所示：

```
1. task 429_R_REG_TC002::run_phase(uvm_phase phase);
2.     automatic logic [31:0] reg0;
3.     automatic logic [31:0] reg1;
4.     automatic logic [31:0] reg2;
5.     automatic logic [31:0] reg3;
6.     automatic logic [31:0] reg4;
7.     automatic logic [31:0] reg9;
8.     automatic logic [31:0] reg10;
9.     automatic logic [31:0] reg11;
10.    phase.raise_objection(this);
11.    $display("429_R_REG_TC002 STARTING...");
12.    tb_system.pcie_read(32'h0000_0000, 0, reg0); // 读取地址 32'h0000_0000 寄存
    器的值
13.    $display("Read data is %x", reg0);
14.    tb_system.pcie_read(32'h0000_0004, 0, reg1);
15.    $display("Read data is %x", reg1);
16.    tb_system.pcie_read(32'h0000_0008, 0, reg2);
17.    $display("Read data is %x", reg2);
18.    tb_system.pcie_read(32'h0000_000C, 0, reg3);
19.    $display("Read data is %x", reg3);
20.    tb_system.pcie_read(32'h0000_0010, 0, reg4);
21.    $display("Read data is %x", reg4);
22.    tb_system.pcie_read(32'h0000_0024, 0, reg9);
23.    $display("Read data is %x", reg9);
24.    tb_system.pcie_read(32'h0000_0028, 0, reg10);
25.    $display("Read data is %x", reg10);
26.    tb_system.pcie_read(32'h0000_002C, 0, reg11);
27.    $display("Read data is %x", reg11);
28.    #20us
29.    phase.drop_objection(this);
30. endtask
```

大语言模型根据手动编写的测试用例结果（如覆盖率报告）生成了更为复杂的寄存器操作测试用例，涵盖了寄存器在极限条件下的表现。具体而言，大语言模型自动模拟了寄存器的高频读写、多次写入与读取之间的同步问题，例如，测试了非法读写、地址冲突等情况，验证了寄存器对这些异常条件的响应能力。

大语言模型生成的用例部分代码如下：

```
1.    tb_system.pcie_read(32'h0000_0000, 0, reg0); // 读取地址 32'h0000_0000 寄存  
    器的值  
2.    $display("Read data is %x", reg0);  
3.    tb_system.pcie_write(32'h0000_0000, 32'hffff_ffff);  
4.    tb_system.pcie_read(32'h0000_0000, 0, reg0); // 读取地址 32'h0000_0000 寄存  
    器的值  
5.    $display("Read data is %x", reg0);
```

通过大语言模型生成的测试用例，进一步验证了寄存器在各种复杂场景下的稳定性和可靠性。模型根据先前的测试反馈动态优化了测试用例，确保了寄存器功能的全面性。这种结合手动测试和大语言模型优化的方式，有效提升了测试覆盖率，并保证了寄存器在各种极端条件下依然能保持高效、稳定的工作性能。

5.3 仿真结果分析

在仿真过程中，对不同的功能模块和测试用例进行了多轮仿真，得到了大量的仿真波形和数据。仿真结果的分析不仅帮助确认验证平台的正确性，还为系统的性能优化提供了依据。本节将依次对数据发送、数据接收、发送配置、接收配置和寄存器基础功能的仿真结果进行分析，并从波形结果和覆盖率分析的角度，全面评估验证平台的性能。

5.3.1 数据发送功能结果分析

在数据发送功能的验证中，首先使用手动编写的测试用例，针对不同波特率和校验方式对发送模块进行了仿真测试。手动测试用例主要验证了在 100Kbps 和 12.5Kbps 波特率下，数据发送模块是否能够按照 ARINC429 协议要求正确发送数据。仿真波形结果如图 5-1 所示，当波特率设置为 100Kbps 时，单比特时间保持在 10000ns，且数据帧的起始位、数据位和停止位严格按照协议规定的时序发送，波形信号稳定且符合预期。数据帧的间隔时间也与协议要求一致，确保数据传输的时序无误。当波特率调整为 12.5Kbps 时，仿真结果同样符合预期，数据位的时序精确，符合协议对波特率的时序要求。

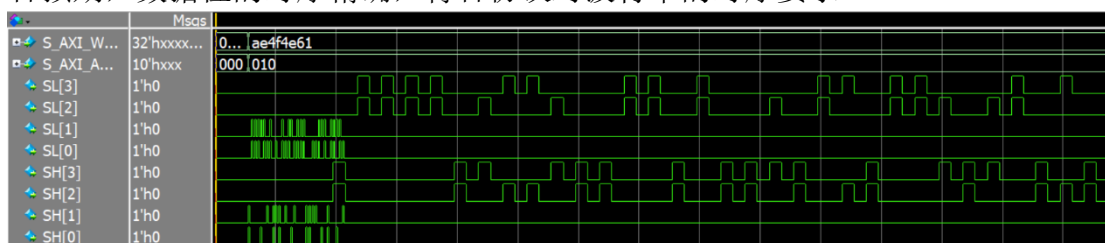


图 5-1 数据发送验证图

大语言模型生成的测试用例进一步扩展了验证场景。这些测试用例基于手动编写的测试用例的覆盖率报告，自动生成了覆盖不足部分的用例。通过模型自动生成的测试用例，验证了更多复杂情况和边界条件，尤其是处理在不同校验方式下的数据传输。模型结合手动测试中未涵盖的场景，通过多轮生成与反馈机制，确保了数据发送模块在不同工作条件下的稳定性和准确性。

5.3.2 数据接收功能结果分析

数据接收功能的仿真分析主要关注接收模块的正确性和稳定性。在这部分的验证中，首先使用手动编写的测试用例，验证了接收模块在 12.5Kbps 波特率下的表现。仿真波形结果如图 5-2 所示，在这种波特率下，接收模块能够成功解码并解析收到的数据帧，数据位、校验位和停止位的时序均严格符合 ARINC429 协议的要求，确保数据的正确接收。

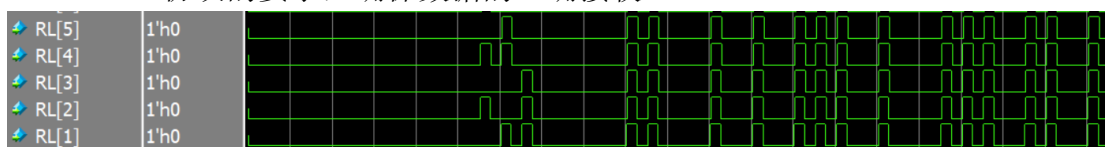


图 5-2 数据接收验证图

大语言模型生成的测试用例则补充了手动测试用例中覆盖不足的部分。基

于手动测试中未能涵盖的场景，大语言模型自动生成了包括接收时序不满足协议要求等复杂情况的测试用例。通过多轮迭代，模型自动调整生成的测试用例，补充了手动测试用例的不足，从而提升了测试覆盖率，确保了接收模块在各种异常情况下的稳定性和正确性。

5.3.3 寄存器基础功能结果分析

寄存器基础功能的仿真验证聚焦于寄存器的读写能力以及在各种操作条件下的稳定性。首先设计了手动编写的测试用例，验证了只写寄存器的写功能。仿真波形如图 5-3 所示。

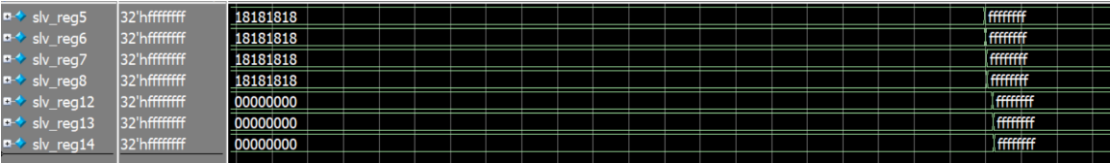


图 5-3 寄存器基础功能验证图

大语言模型生成的测试用例进一步扩展了手动测试用例的覆盖范围，尤其是在寄存器高频读写操作下的表现。通过大语言模型自动生成的测试用例，能够模拟更多复杂场景，包括连续的高频读写操作、不同数据的快速存取等。大语言模型根据手动测试用例的覆盖率分析结果，补充了手动测试未涵盖的极限情况。仿真结果表明，寄存器能够在这些高频操作条件下保持稳定，准确地存储和读取数据，没有出现数据丢失或错误存取的情况。这不仅验证了寄存器的高频操作能力，还提高了测试覆盖率，确保了寄存器功能的全面性与稳定性。

5.4 覆盖率分析

在本次 ARINC429 协议验证中，覆盖率是衡量验证工作完备性的重要指标。通过多轮仿真和测试用例的执行，对代码覆盖率和功能覆盖率进行了详细的分析。最初，在使用手动编写的测试用例进行验证时，虽然覆盖了基本的功能点，但测试覆盖率仍然不足。因此，在这部分验证中，引入了基于大语言模型的测试用例生成策略，以进一步优化覆盖率，确保验证的全面性和准确性。

在初始阶段，手动编写的测试用例覆盖了基本的功能模块，但由于人工编写的测试用例有限，部分复杂场景和边界条件没有得到充分的验证。这导致了初始验证结果中的代码覆盖率和功能覆盖率存在不足，验证效果较为薄弱。初步的测试结果显示，代码覆盖率和功能覆盖率未能完全覆盖设计的所有功能点和代码路径，如图 5-4 所示。

Coverage Summary By Instance:

Scope ▾	TOTAL ▾	Statement ▾	Branch ▾	Condition ▾
TOTAL	77.49	85.67	81.52	65.29
inst	77.49	85.67	81.52	65.29

Recursive Hierarchical Coverage Details:

Total Coverage:					77.49%
Coverage Type ▾	Bins ▾	Hits ▾	Misses ▾	Weight ▾	Coverage ▾
Statements	10192	8732	1460	1	85.67%
Branches	5617	4579	1038	1	81.52%
Conditions	1870	1221	649	1	65.29%

图 5-4 初始覆盖率

通过使用大语言模型生成的测试用例，显著提升了覆盖率。大语言模型基于手动编写的测试用例结果，分析了覆盖率不足的部分，并自动生成了能够填补这些空白的测试用例。经过大语言模型的一轮互动后，测试用例覆盖了更多的功能路径，确保了测试的全面性和深度。这些提升表明，经过大语言模型生成的测试用例补充后，验证过程对设计的各个功能点和代码路径进行了更加全面的覆盖，有效减少了验证中的遗漏。

通过反馈机制的作用，大语言模型在每一轮的互动中，基于覆盖率和执行结果不断优化生成的测试用例，进一步提高了测试覆盖率并完善了测试内容。在最终的测试结果中，代码覆盖率和功能覆盖率都达到了 100%，如图 5-5 所示，这意味着本文的测试用例成功地覆盖了设计中的所有关键功能和代码路径，验证工作已达到了预期的全面性和准确性。

Coverage Summary By Instance:

Scope ▾	TOTAL ▾	Statement ▾	Branch ▾	Condition ▾
TOTAL	100.00	100.00	100.00	100.00
inst	100.00	100.00	100.00	100.00

Recursive Hierarchical Coverage Details:

Total Coverage:					100.00%
Coverage Type ▾	Bins ▾	Hits ▾	Misses ▾	Weight ▾	Coverage ▾
Statements	10192	10192	0	1	100.00%
Branches	5617	5617	0	1	100.00%
Conditions	1870	1870	0	1	100.00%

图 5-5 最终覆盖率

综上所述，通过大语言模型生成的测试用例与手动编写的测试用例相结合，不仅提升了覆盖率，也优化了验证效率和准确性。最终，验证平台在多轮回归测试后，成功达到了 100%的代码覆盖率和功能覆盖率，保证了 ARINC429 协议模块的功能正确性和稳定性。

5.5 本章小结

本章介绍了基于 UVM 的 ARINC429 协议验证平台的仿真验证与结果分析。通过多轮仿真验证，验证了数据发送、接收、配置和寄存器功能的正确性。结合手动编写的测试用例和大语言模型生成的测试用例，显著提升了验证的覆盖率和全面性。

覆盖率分析表明，初始覆盖率不足的情况下，大语言模型生成的测试用例通过一轮互动后大幅提升了覆盖率，最终达到了 100%的代码和功能覆盖率。这表明大语言模型在提高验证效率和确保设计全面性方面的有效性。

第6章 总结与展望

6.1 工作总结

本研究围绕基于 UVM 的 ARINC429 模块验证平台设计与实现展开，针对航空电子系统中 ARINC429 协议的验证需求，通过结合 UVM 方法学与大语言模型优化验证过程，提出并实现了高效且全面的验证方案。本文的研究工作具体体现在以下几个方面：

（1）基于大语言模型的测试用例生成方法

传统的验证方法通常依赖人工生成测试用例，这样的方式在面对复杂协议时效率较低，且覆盖面不足。为了弥补这一缺陷，本文提出了一种基于大语言模型的测试用例生成方法。通过将设计代码和覆盖率报告输入大语言模型，并利用自定义的提示模板进行引导，生成精准的测试用例。这一方法不仅提高了测试用例生成的效率，还能够动态调整生成策略，确保验证过程的全面性和有效性。通过实验验证，在完成相同验证目标下的，相较于基于随机约束生成的测试用例，基于大语言模型生成的用例显著提高了验证覆盖率，具体的，语句覆盖率提升了 8.33%，分支覆盖率提升了 9.45%，条件覆盖率提升了 8.05%，并且验证时间缩短了约 70%，从 4 小时减少至 1.3 小时。

（2）基于 UVM 的 ARINC429 模块验证平台设计与实现

基于测试用例生成方法的基础，本文成功搭建了一个基于 UVM 的 ARINC429 模块验证平台。该平台通过结合自动生成的测试用例，利用覆盖率反馈机制实现自我优化和动态调整，从而确保 ARINC429 协议在不同功能层面的全面验证。平台的自动化程度较高，验证效率显著提高，同时由于优化过的测试用例，验证过程的准确性和全面性得到了保障。该平台不仅能够有效处理 ARINC429 模块的验证任务，还为类似协议的验证提供了可借鉴的解决方案。

（3）ARINC429 模块仿真验证与结果分析

在 ARINC429 模块设计和验证平台搭建完成后，本文进行了仿真验证，并对仿真结果进行了详细分析。通过对不同测试用例的仿真，重点分析了数据发

送、接收、配置功能等模块的正确性及稳定性。结合大语言模型生成的测试用例，进一步扩展了验证范围，涵盖了更多极限条件和边界场景。通过仿真波形与覆盖率分析，验证了 ARINC429 模块的功能是否满足设计要求，最终通过多轮回归测试，确保了功能和代码覆盖率达到 100%。

6.2 未来展望

尽管本文在 ARINC429 协议的验证工作中取得了显著的成果，但仍然存在一些不足之处，未来的研究工作可从以下几个方面进行进一步的优化和扩展：

（1）从模块级验证扩展到系统级验证

本研究着重围绕 ARINC429 模块的功能验证展开，验证范畴限定于模块级别。但是模块级验证很难囊括模块间交互以及系统级性能等关键部分，在验证视角与验证深度方面存在一定局限。未来的研究可扩展到系统级验证，着重剖析模块级验证和系统级验证的集成途径。例如可以搭建系统级仿真环境，把多个协议模块整合到复杂应用场景里，模拟模块间的协同与交互行为，依靠引入系统性能评估指标，对系统在高负载、异常场景下的稳定性、可靠性以及一致性进行验证。此外，可结合硬件在环平台，把仿真和实际硬件相结合，提高验证的真实性以及覆盖广度，逐步达成从单模块验证到系统级验证的平稳过渡，提升航空电子系统验证的整体效率与质量。

（2）大语言模型与验证流程的深度融合

未来研究可拓宽大语言模型的应用范畴，除持续优化验证流程之外，还可将其运用于设计代码以及验证平台的自动生成方面。借助引入大语言模型生成设计代码，可提高开发效率，而且可以保证代码的一致性与质量，模型可依据验证需求生成适配的验证平台，缩减手动搭建验证环境的时间，提升验证过程的自动化程度。如此一来，大语言模型可提高整个验证流程的智能化水准，推动验证技术向前发展。

（3）生成式语言模型局限性分析与优化

虽然本文所提出的借助生成式语言模型的测试用例生成方法，在提高验证效率方面有一定优势呈现出来，可是它在实际运用过程里依旧存在一些局限性。例如，生成用例的准确性和稳定性有可能会受到模型上下文理解能力的约束，

在航空电子等有高可靠性、高安全性的领域，容易出现用例生成覆盖不全面或者遗漏边界场景的情况。另外当前生成过程主要依靠静态提示，缺少和验证平台的动态反馈，很难达成基于验证覆盖率以及仿真结果的自适应优化。

在未来的工作当中，可强化对于生成式语言模型局限性的系统性剖析，依靠结合领域知识、规则库以及测试经验，建立起自动化的用例筛选与验证机制，以此提升生成用例的有效性以及严谨性。

参考文献

- [1] 曹冬冬,来鑫雪,宋承斌,等.我国汽车芯片发展现状及趋势分析[J].中国汽车,2024,(11):15-21+64.
- [2] 刘岩,刘妍,张如意,等.会聚视角下人工智能芯片领域关键核心技术发展态势与突破路径研究[J].科技管理研究,2024,44(19):11-23.
- [3] 宋艳飞,孙佳琪,王睿哲.人工智能芯片技术演进与发展策略研究[J].信息技术与标准化,2024,(06):94-98.
- [4] Wang G, Zhao W. The principles of integrated technology in avionics systems[M]. Academic Press, 2020.
- [5] de Almeida A A, Saotome¹ O. Reliability Guaranteed ARINC 429[C]//Proceedings of the 9th Brazilian Technology Symposium (BTSym'23): Emerging Trends and Challenges in Technology. Springer Nature, 2024, 402: 140.
- [6] Kim B K, Cha Y H. The Proposal Method of ARINC-429 Linkage for Efficient Operation of Tactical Stations in P-3C Maritime Patrol Aircraft[J]. Journal of Advanced Navigation Technology, 2023, 27(2): 167-172.
- [7] 李波,姚金彪.基于机载 ARINC429 总线输出的纬度差数据异常判定与处理[J].中国军转民,2024,(01):11-12.
- [8] 潘硕,徐婕,任杰,等.ARINC429 航空总线典型器件与接口设计[J].电子制作,2023,31(12):82-84+92.DOI:10.16589/j.cnki.cn11-3571/tn.2023.12.022.
- [9] Gao Y, Zhang Z, Wang J, et al. Research on the meteorological data transmission system of civil aircraft based on BeiDou satellite[C]//Journal of Physics: Conference Series. IOP Publishing, 2024, 2879(1): 012059.
- [10] 宋丫,李兴智,刘逸涵.机载通信 ARINC429 总线协议总线心跳故障案例分析[J].山西电子技术,2021,(05):29-31.
- [11] 张晋旗.基于 UVM 的 AHB MATRIX 验证平台设计[D].杭州电子科技大学,2024.DOI:10.27075/d.cnki.ghzdc.2024.001413.
- [12] Bhandari J, Knechtel J, Narayanaswamy R, et al. LLM-aided testbench generation and bug detection for finite-state machines[J]. arXiv preprint arXiv:2406.17132,

2024.

- [13] Reddy K V. UVM Methodology: Industry-Specific Applications in Modern Hardware Verification[J]. International Journal of Computer Engineering and Technology (IJCET), 2024, 15(6): 20-32.
- [14] Gamboa A B, Chipres C, Jadhav S. UVM-Based Design and Verification of AHB-Lite to AXI Bridge[C]//SoutheastCon 2024. IEEE, 2024: 472-479.
- [15] Liao C W, Yu H C, Liao Y C. Verification of SPI Protocol Using Universal Verification Methodology for Modern IoT and Wearable Devices[J]. Electronics, 2025, 14(5): 837.
- [16] Prasad R A, Kulkarni M. Design and verification of phy interface for pcie gen 3.0 and usb gen 3.1 using UVM methodology[J]. International Research Journal of Engineering and Technology (IRJET), 2017, 4(10).
- [17] Kamaraju M, Tilak A V N, Kishore K L, et al. VHDL Implementation and Verification of ARINC-429 Core[J]. arXiv preprint arXiv:1011.5374, 2010.
- [18] Liqin Y, Xiaobai L, Hui L. Simulation of ARINC429 bus signal and analysis of transmission characteristics[C]//2010 International Conference on Electrical and Control Engineering. IEEE, 2010: 1919-1922.
- [19] Ayaz İ, Sakarya U, Hökelek İ. Universal verification methodology application of ARINC-429 for airborne electronic hardware certification[C]//2021 13th International Conference on Electrical and Electronics Engineering (ELECO). IEEE, 2021: 162-166.
- [20] 阳一涵.基于 UVM 的 ARINC429 总线接口模块验证平台的设计与实现[D].西安电子科技大学,2019.DOI:10.27389/d.cnki.gxadu.2019.002354.
- [21] 李屹爽.基于 UVM 验证方法学的 APB-ARINC429 模块设计与验证[D].西安建筑科技大学,2024.
- [22] Zhong R, Du X, Kai S, et al. Llm4eda: Emerging progress in large language models for electronic design automation[J]. arXiv preprint arXiv:2401.12224, 2023.
- [23] Liu M, Pinckney N, Khailany B, et al. VerilogEval: Evaluating large language models for verilog code generation[C]//2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD). IEEE, 2023: 1-8.

-
- [24] Thakur S, Blocklove J, Pearce H, et al. Autochip: Automating hdl generation using llm feedback[J]. arXiv preprint arXiv:2311.04887, 2023.
- [25] Lu Y, Liu S, Zhang Q, et al. Rtlm: An open-source benchmark for design rtl generation with large language model[C]//2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC). IEEE, 2024: 722-727.
- [26] Thakur S, Ahmad B, Pearce H, et al. Verigen: A large language model for verilog code generation[J]. ACM Transactions on Design Automation of Electronic Systems, 2024, 29(3): 1-31.
- [27] Kande R, Pearce H, Tan B, et al. Llm-assisted generation of hardware assertions[J]. arXiv preprint arXiv:2306.14027, 2023.
- [28] Yan Z, Fang W, Li M, et al. Assertllm: Generating and evaluating hardware verification assertions from design specifications via multi-llms[J]. arXiv preprint arXiv:2402.00386, 2024.
- [29] Blocklove J, Garg S, Karri R, et al. Chip-chat: Challenges and opportunities in conversational hardware design[C]//2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD). IEEE, 2023: 1-6.
- [30] Fu Y, Zhang Y, Yu Z, et al. Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models[C]//2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD). IEEE, 2023: 1-9.
- [31] Chang K, Wang Y, Ren H, et al. ChipGPT: How far are we from natural language hardware design[J]. arXiv preprint arXiv:2305.14019, 2023.
- [32] Wu H, He Z, Zhang X, et al. Chateda: A large language model powered autonomous agent for eda[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2024.
- [33] Bergeron J, Cerny E, Hunter A, et al. Verification methodology manual for SystemVerilog[M]. New York: Springer, 2006.
- [34] 刘斌.芯片验证漫游指南[M].电子工业出版社:201804.560.
- [35] Ni W, Zhang J. Research of reusability based on UVM verification[C]//2015 IEEE 11th International Conference on ASIC (ASICON). IEEE, 2015: 1-4.
- [36] Melikyan V, Harutyunyan S, Kirakosyan A, et al. Uvm verification ip for axi[C]//2021 IEEE East-West Design & Test Symposium (EWDTS). IEEE, 2021:

1-4.

- [37] Pavithran T M, Bhakthavatchalu R. UVM based testbench architecture for logic sub-system verification[C]//2017 International Conference on Technological Advancements in Power and Energy (TAP Energy). IEEE, 2017: 1-5.
- [38] Gamboa A B, Chipres C, Jadhav S. UVM-Based Design and Verification of AHB-Lite to AXI Bridge[C]//SoutheastCon 2024. IEEE, 2024: 472-479.
- [39] Kumar P S, Rajalakshmi R, Kumar N H, et al. Robust Serial Driver Verification Through UVM Framework[C]//2024 Control Instrumentation System Conference (CISCON). IEEE, 2024: 1-6.
- [40] Liao C W, Yu H C, Liao Y C. Verification of SPI Protocol Using Universal Verification Methodology for Modern IoT and Wearable Devices[J]. Electronics, 2025, 14(5): 837.
- [41] Dargar A, Naidu M L, Karthik S, et al. Development of Serial Driver Verification Environment Module Using UVM Method[C]//2024 5th International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV). IEEE, 2024: 535-540.
- [42] Ramesh M, Kumar D C, Yashwanth L, et al. Five Stage Pipelined MIPS Processor Verification Interface and Test Module using UVM[C]//2023 International Conference on Sustainable Computing and Smart Systems (ICSCSS). IEEE, 2023: 1226-1232.
- [43] Sahay N, Gajjar S. Design and UVM based Verification of UART, SPI, and I 2 C Protocols[C]//2024 5th International Conference on Smart Electronics and Communication (ICOSEC). IEEE, 2024: 330-335.
- [44] Wang D, Yan J, Qiao Y. Research on Chip Verification Technology Based on UVM[C]//2021 6th International Symposium on Computer and Information Processing Technology (ISCIPT). IEEE, 2021: 117-120.
- [45] Bhatt P, Joshi D, Jadhav S. RTL to GDS Implementation and Verification of UART using UVM and OpenROAD[C]//2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC). IEEE, 2024: 0713-0720.
- [46] 谢峥,王腾,雍珊珊,等.一种基于 UVM 面向 RISCCPU 的可重用功能验证平台[J].北京大学学报(自然科学版),2014,50(02):221-227.

- [47] 高秋辰,胡勇华.基于 UVM 的 SoC 环境中 PCIe 验证平台设计[J].计算机工程,2024,50(09):189-196.
- [48] Pothana N, Reddy V H P. Verification of Coverage-Driven and Assertion based AXI4 Protocol VIP using UVM[C]//2023 First International Conference on Cyber Physical Systems, Power Electronics and Electric Vehicles (ICPEEV). IEEE, 2023: 1-7.
- [49] Vitankar P S, Kureshi A K. UVM architecture for verification[J]. International Journal of Electronics and communications Engineering and Technology, 2016, 7(3): 29-37.
- [50] Vencato L. Development of System Verification Methods for Aircraft Electronic Systems[D]. Politecnico di Torino, 2024.
- [51] Strobelt H, Webson A, Sanh V, et al. Interactive and visual prompt engineering for ad-hoc task adaptation with large language models[J]. IEEE transactions on visualization and computer graphics, 2022, 29(1): 1146-1156.
- [52] Abukhalaf S, Hamdaqa M, Khomh F. On Codex prompt engineering for OCL generation: an empirical study[C]//2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR). IEEE, 2023: 148-157.
- [53] Oppenlaender J, Linder R, Silvennoinen J. Prompting AI art: An investigation into the creative skill of prompt engineering[J]. International Journal of Human-Computer Interaction, 2024: 1-23.
- [54] Chalkidis I. Chatgpt may pass the bar exam soon, but has a long way to go for the lexglue benchmark[J]. arXiv preprint arXiv:2304.12202, 2023.
- [55] Zhan T, Shi C, Shi Y, et al. Optimization Techniques for Sentiment Analysis Based on LLM (GPT-3)[J]. arXiv preprint arXiv:2405.09770, 2024.
- [56] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.
- [57] Achiam J, Adler S, Agarwal S, et al. Gpt-4 technical report[J]. arXiv preprint arXiv:2303.08774, 2023.
- [58] Devlin J. Bert: Pre-training of deep bidirectional transformers for language understanding[J]. arXiv preprint arXiv:1810.04805, 2018.
- [59] Raffel C, Shazeer N, Roberts A, et al. Exploring the limits of transfer learning with

- a unified text-to-text transformer[J]. Journal of machine learning research, 2020, 21(140): 1-67.
- [60] Chowdhery A, Narang S, Devlin J, et al. Palm: Scaling language modeling with pathways[J]. Journal of Machine Learning Research, 2023, 24(240): 1-113.
- [61] Zhang S, Roller S, Goyal N, et al. Opt: Open pre-trained transformer language models[J]. arXiv preprint arXiv:2205.01068, 2022.
- [62] Oppenlaender J. A taxonomy of prompt modifiers for text-to-image generation[J]. Behaviour & Information Technology, 2023: 1-14.
- [63] White J, Fu Q, Hays S, et al. A prompt pattern catalog to enhance prompt engineering with chatgpt[J]. arXiv preprint arXiv:2302.11382, 2023.
- [64] Lo L S. The CLEAR path: A framework for enhancing information literacy through prompt engineering[J]. The Journal of Academic Librarianship, 2023, 49(4): 102720.
- [65] Short C E, Short J C. The artificially intelligent entrepreneur: ChatGPT, prompt engineering, and entrepreneurial rhetoric creation[J]. Journal of Business Venturing Insights, 2023, 19: e00388.
- [66] Strobelt H, Webson A, Sanh V, et al. Interactive and visual prompt engineering for ad-hoc task adaptation with large language models[J]. IEEE transactions on visualization and computer graphics, 2022, 29(1): 1146-1156.
- [67] Abukhalaf S, Hamdaqa M, Khomh F. On Codex prompt engineering for OCL generation: an empirical study[C]//2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR). IEEE, 2023: 148-157.
- [68] Oppenlaender J, Linder R, Silvennoinen J. Prompting AI art: An investigation into the creative skill of prompt engineering[J]. International Journal of Human–Computer Interaction, 2024: 1-23.
- [69] Chalkidis I. Chatgpt may pass the bar exam soon, but has a long way to go for the lexglue benchmark[J]. arXiv preprint arXiv:2304.12202, 2023.

附录

本文采用的硬件和软件环境配置如下：

1. 硬件环境

计算机型号：神州 Z6-KP5GT

处理器：Intel Core i5-7300HQ 四核处理器

内存：8GB DDR4

存储：128 SSD + 1TB HDD

2. 软件环境

操作系统：Ubuntu

仿真工具：Questa Sim 10.7c

验证平台：UVM

大语言模型：OpenAI GPT-3.5-turbo

编程语言：Verilog、SystemVerilog、Python

本文采用的部分用例代码：

寄存器只读属性测试用例代码：

```
1. `ifndef 429_R_REG_TC002__SV
2. `define 429_R_REG_TC002__SV
3. import my_utils_pkg::*;
4.
5. class 429_R_REG_TC002 extends base_test;
6.     function new(string name = "429_R_REG_TC002", uvm_component parent =
       null);
7.         super.new(name, parent);
8.     endfunction
9.     extern virtual function void build_phase(uvm_phase phase);
10.    extern virtual task run_phase(uvm_phase phase);
11.    `uvm_component_utils(429_R_REG_TC002);
12.    pcie_429_sequence pcie_429_seq; // 基本测试序列
13.    arinc429_config arinc429_cfg; // 全局配置对象
14. endclass
15.
16. function void 429_R_REG_TC002::build_phase(uvm_phase phase);
```

```
17. super.build_phase(phase);
18. // 初始化全局配置对象
19. arinc429_cfg = arinc429_config::new("arinc429_cfg");
20. arinc429_cfg.monitor_rate = 1; // 设置参数值
21. // 将全局配置对象放入配置数据库
22. uvm_config_db#(arinc429_config)::set(this, "*", "arinc429_cfg",
    arinc429_cfg);
23. endfunction
24.
25. task 429_R_REG_TC002::run_phase(uvm_phase phase);
26. automatic logic [31:0] reg0;
27. automatic logic [31:0] reg1;
28. automatic logic [31:0] reg2;
29. automatic logic [31:0] reg3;
30. automatic logic [31:0] reg4;
31. automatic logic [31:0] reg9;
32. automatic logic [31:0] reg10;
33. automatic logic [31:0] reg11;
34. phase.raise_objection(this);
35. $display("429_R_REG_TC002 STARTING...");
36. tb_system.pcie_read(32'h0000_0000, 0, reg0); // 读取地址 32'h0000_0000 寄存
    器的值
37. $display("Read data is %x", reg0);
38. tb_system.pcie_read(32'h0000_0004, 0, reg1);
39. $display("Read data is %x", reg1);
40. tb_system.pcie_read(32'h0000_0008, 0, reg2);
41. $display("Read data is %x", reg2);
42. tb_system.pcie_read(32'h0000_000C, 0, reg3);
43. $display("Read data is %x", reg3);
44. tb_system.pcie_read(32'h0000_0010, 0, reg4);
45. $display("Read data is %x", reg4);
46. tb_system.pcie_read(32'h0000_0024, 0, reg9);
47. $display("Read data is %x", reg9);
48. tb_system.pcie_read(32'h0000_0028, 0, reg10);
49. $display("Read data is %x", reg10);
50. tb_system.pcie_read(32'h0000_002C, 0, reg11);
51. $display("Read data is %x", reg11);
52. #20us
53. phase.drop_objection(this);
54. endtask
55.
56. `endif
```

数据发送功能测试用例代码:

```
1. `ifndef 429_TC043__SV
2. `define 429_TC043__SV
3. import my_utils_pkg::*;
4.
5. class 429_TC043 extends base_test;
6.     function new(string name = "429_TC043", uvm_component parent = null);
7.         super.new(name, parent);
8.     endfunction
9.     extern virtual function void build_phase(uvm_phase phase);
10.    extern virtual task run_phase(uvm_phase phase);
11.    `uvm_component_utils(429_TC043);
12.    pcie_429_sequence pcie_429_seq; // 基本测试序列
13.    arinc429_config arinc429_cfg; // 全局配置对象
14. endclass
15.
16. function void 429_TC043::build_phase(uvm_phase phase);
17.     super.build_phase(phase);
18.     // 初始化全局配置对象
19.     arinc429_cfg = arinc429_config::new("arinc429_cfg");
20.     arinc429_cfg.monitor_rate = 1; // 设置参数值
21.     // 将全局配置对象放入配置数据库
22.     uvm_config_db#(arinc429_config)::set(this, "*", "arinc429_cfg",
        arinc429_cfg);
23. endfunction
24.
25. task 429_TC043::run_phase(uvm_phase phase);
26.     phase.raise_objection(this);
27.     $display("429_TC043 STARTING...");
28.     //第 1 路到第 4 路发送通道奇偶校验 波特率
29.     tb_system.pcie_write(32'h0000_1014, 32'h1018_0008);
30.     //第 1 路发送通道 100kbps 偶校验
31.     tb_system.pcie_write(32'h0000_1000, 32'h2518_4511); //预期 100us 出结果
32.     //第 2 路发送通道 100kbps 奇校验
33.     tb_system.pcie_write(32'h0000_1004, 32'h8478_9741); //预期 100us 出结果
34.     //第 3 路发送通道 12.5kbps 偶校验
35.     tb_system.pcie_write(32'h0000_1008, 32'h646b_4da1); //预期 400us 出结果
36.     //第 4 路发送通道 12.5kbps 奇校验
37.     tb_system.pcie_write(32'h0000_100C, 32'hae4f_4e61); //预期 400us 出结果
38.     #3ms phase.drop_objection(this);
39. endtask
```

```

40.
41. `endif

```

数据接收功能测试用例代码:

```

1. `ifndef 429_TC049__SV
2. `define 429_TC049__SV
3. import my_utils_pkg::*;
4.
5. class IO_TC049 extends base_test;
6.     function new(string name = "IO_TC049", uvm_component parent = null);
7.         super.new(name, parent);
8.     endfunction
9.     extern virtual function void build_phase(uvm_phase phase);
10.    extern virtual task run_phase(uvm_phase phase);
11.    `uvm_component_utils(429_TC049);
12.    pcie_429_sequence pcie_429_seq; // 基本测试序列
13.    arinc429_config arinc429_cfg; // 全局配置对象
14. endclass
15.
16. function void 429_TC049::build_phase(uvm_phase phase);
17.     super.build_phase(phase);
18.     // 初始化全局配置对象
19.     arinc429_cfg = arinc429_config::new("arinc429_cfg");
20.     arinc429_cfg.monitor_rate = 1; // 设置参数值
21.     // 将全局配置对象放入配置数据库
22.     uvm_config_db#(arinc429_config)::set(this, "*", "arinc429_cfg",
        arinc429_cfg);
23. endfunction
24.
25. task 429_TC049::run_phase(uvm_phase phase);
26.     automatic logic [31:0] recv_buf_status = 32'b0;
27.     automatic logic [31:0] receive_data = 32'b0;
28.     phase.raise_objection(this);
29.     $display("429_TC049 STARTING...");
30.
31.     //发送通道配置 偶校验 100kbps slv_reg32 前四发送通路配置
32.     tb_system.pcie_write(32'h0000_0014, 32'h1010_1010);
33.     //发送数据 4通路 slv_reg0 - slv_reg3
34.     for (int i = 0; i < 4; i++) begin
35.         tb_system.pcie_write(32'h0000_1000 + i * 4, 32'h1234_5678 + i);
36.     end

```

```
37. //接收通道配置 偶校验 前4路接收通路配置
38. tb_system.pcie_write(32'h0000_001C, 32'h1010_1010);
39.
40. //等待前4通路有数据 接收FIFO缓冲状态
41. wait(tb_system.u_top.system_user.system_i.ARINC429_0.inst.ARINC429_v1_0_S_
    AXI_inst.recv_buf_status==24'h0000ff);
42.
43. tb_system.pcie_read(32'h0000_0028, 0, recv_buf_status);
44.
45. for (int i = 0; i < 4; i++) begin
46.     if(recv_buf_status[i]) begin
47.         tb_system.pcie_write(32'h0000_0034, 32'h5500_0000 + i);
48.         tb_system.pcie_read(32'h0000_0000 + i * 4, 0, receive_data);
49.         $display("channel %d is not empty",i);
50.         $display("channel %d receive data is %x", i, receive_data);
51.     end
52. end
53. #100us phase.drop_objection(this);
54. endtask
55.
56. `endif
```